TECHNISCHE HOCHSCHULE DEGGENDORF

Kurs: Digitale Forensik

Teammitglieder: Shahed Al Hendi (22300760)

Janine Heinl (12306658)

Abdusalam Alhammoud (00817079)

Betreuer: Prof. Dr. Michael Heigl

Thema: Forensische Analyse an einem fiktiven Fall

Fall Nr. 4321 – Entführung Rosanna Lorenzo

Inhaltsverzeichnis

1 Einleitung ... 1
1.1 Zielsetzung ... 1
1.2 Arbeitsaufteilung ... 2
1.3 Aufbau des Berichts ... 2
2 Sachverhalt ... 2
3 Vorbereitete digitale Spuren ... 3
3.1 Chatverlauf über die Dating-Plattform (parship_chatverlauf.json) ... 3
3.2 Standort-Tracker und Standort-Log (Tracker.py, Locations.log) ... 3
3.3 Entführungsplan als Steganografie (plan.jpg) ... 4
3.4 Standort des Verstecks über EXIF-Metadaten (lager.webp) ... 4
3.5 Bitcoin-Wallet (wallet.dat) – Vorbereitung ... 4
3.6 USB-Container mit weiteren Beweisdateien ... 5
4 Asservate und Chain of Custody ... 5
4.1 Übersicht der Asservate ... 5
4.2 Verwendete Untersuchungsumgebung ... 7
4.3 Chain of Custody ... 7
5 Erkennung – Warum der Verdacht auf Simon Aman fiel ... 8
6 Initial Response (Erstzugriff am Tatort) ... 8
7 Methodik und eingesetzte Werkzeuge ... 10
7.1 Eingesetzte Software ... 10
7.2 Werkzeuge im Detail ... 10
7.3 Untersuchungsablauf ... 11
7.4 Wahl des Image-Formats (Raw/dd) ... 11
8 Beweissicherung – Images und Hashwerte ... 12
8.1 Integritätsprüfung mittels Hashwerten ... 12
8.2 Übersicht der erzeugten Images ... 12
8.3 Übersicht der Beweismittel ... 12
9 Auswertung ... 13
9.1 Methodik und Untersuchungsansatz ... 13
9.2 Systemidentifikation und Dateisystemstruktur ... 13
9.3 Untersuchung des Benutzerprofils ... 14
9.3.1 Auswertung des Shell-Verlaufs (.bash_history) ... 14
9.3.2 Auswertung geplanter Aufgaben (Cron) ... 16

9.3.3 Auswertung zuletzt verwendeter Dateien (recently-used.xbel) 17
9.4 Wiederherstellung gelöschter Dateien 17
9.5 Chat-Verlauf (parship_chatverlauf.json) 18
9.6 Standort-Tracking (tracker.py / Locations.log) 19
9.7 EXIF-Metadaten (lager.webp) 20
9.8 Bitcoin-Wallet (wallet.dat) 21
9.9 Arbeitsspeicheranalyse und Rückgewinnung des Kontopassworts 21
9.10 Steganografie (plan.jpg) 23
9.11 USB-Container – Entschlüsselung (LUKS) 24
9.12 Verwendete Werkzeuge und Untersuchungsumgebung 26
9.13 Beweiskette und Ergebnis 27
10 Zeitachse 27
11 Beweiskette – Zusammenfassung 28
12 Rechtliche Einordnung 28
13 Fazit 28
13.1 Grenzen der Untersuchung und Reproduzierbarkeit 29
13.2 Einschränkungen und Ausblick 29
14 Quellenverzeichnis 30
Abkürzungsverzeichnis 30
15 Anhänge 30
Anhang A (Nextcloud) 30

1 Einleitung

Im Rahmen dieser Projektarbeit wurde ein fiktiver kriminalistischer Fall entwickelt und anschließend mit Methoden der digitalen Forensik untersucht. Ziel der Projektarbeit war es, einen möglichst realitätsnahen Untersuchungsablauf nachzubilden, von der Vorbereitung digitaler Beweismittel über die Erstellung forensischer Datenträgerabbilder bis hin zur späteren Auswertung der gesicherten Artefakte.

Als Fallbeispiel wurde die fiktive Entführung der 15-jährigen Rosanna Lorenzo gewählt. Im Mittelpunkt der Untersuchung steht der Tatverdächtige Simon Aman, auf dessen virtuellem System verschiedene digitale Spuren vorbereitet wurden. Hierzu zählen unter anderem gelöschte Dateien, ein verschlüsselter USB-Container, ein Arbeitsspeicherabbild sowie weitere digitale Artefakte. Diese bilden die Grundlage für die anschließende forensische Analyse und ermöglichen den Einsatz verschiedener Methoden der Festplatten- und Speicherforensik.

Die vorliegende Dokumentation beschreibt zunächst die Vorbereitung des Falls, die Erstellung der Beweismittel sowie die Maßnahmen zur Integritätssicherung. Anschließend werden die Ergebnisse der forensischen Untersuchung dokumentiert und hinsichtlich ihres Beweiswertes eingeordnet.

1.1 Zielsetzung

Ziel dieser Projektarbeit war die Entwicklung eines realitätsnahen forensischen Falls, bei dem unterschiedliche Methoden der digitalen Forensik praktisch angewendet werden können. Hierzu wurden gezielt digitale Artefakte auf einem virtuellen Ubuntu-System sowie einem verschlüsselten USB-Container vorbereitet. Ein Teil der Dateien wurde bewusst gelöscht oder verborgen, um typische Untersuchungsszenarien wie die Wiederherstellung gelöschter Daten, die Analyse eines Arbeitsspeicherabbildes, die Auswertung von Metadaten sowie die Untersuchung steganografischer Inhalte nachzustellen.

Neben der technischen Umsetzung des Falls bestand ein weiteres Ziel darin, den gesamten Ablauf einer digitalen Untersuchung nachvollziehbar zu dokumentieren. Dazu gehören sowohl die Erstellung forensischer Images einschließlich der Integritätsprüfung mittels Hashwerten als auch die spätere Auswertung der gesicherten Beweismittel.

1.2 Arbeitsaufteilung

Die Aufteilung der Arbeitsleistung innerhalb des Teams ist in der untenstehenden Tabelle dargestellt.

Teilaufgabe	**Bearbeiter/in**	**Anteil**
Fall konzipieren, Artefakte erstellen, VMs/Images vorbereiten	Shahed Al Hendi	40 %
Forensische Analyse (Wiederherstellung, RAMAuswertung, Steganografie, Metadaten etc.)	Abdusalam Alhammoud	40 %
Dokumentation / Bericht	Janine Heinl	20 %

1.3 Aufbau des Berichts

Der Bericht beginnt mit der Beschreibung des zugrunde liegenden Sachverhalts sowie der Vorbereitung der digitalen Beweismittel. Anschließend werden die sichergestellten Asservate, die verwendeten Werkzeuge und die Maßnahmen zur Integritätssicherung erläutert. Darauf aufbauend werden die Ergebnisse der forensischen Untersuchung dargestellt und hinsichtlich ihrer Bedeutung für den vorliegenden Fall bewertet. Abschließend erfolgt eine Zusammenfassung der gewonnenen Erkenntnisse sowie eine Einordnung der Grenzen der Untersuchung.

2 Sachverhalt

Feld	**Wert**
Fallnummer	4321
Verdächtiger	Simon Aman
Opfer	Rosanna Lorenzo (15 Jahre)
Tatvorwurf	Entführung, Erpressung

Tatzeitraum	Februar – März 2026

Kurz zusammengefasst: Der Verdächtige nimmt über eine Dating-Plattform unter einem Pseudonym Kontakt zum Opfer auf. Über etwa eine Woche entsteht ein Kontakt, der in einem heimlich verabredeten Treffen am Abend des Geburtstags des Opfers endet. Danach bricht die Standortspur des Opfer-Handys ab. Die Ermittlungshypothese geht davon aus, dass das Opfer zur Wohnung des Verdächtigen und von dort in eine leerstehende Lagerhalle gebracht wurde. Auf eine wörtliche Wiedergabe der Chat-Inhalte wird hier bewusst verzichtet. Für die Bewertung als Beweismittel reicht die Zusammenfassung von Ablauf und Zeitpunkten völlig aus. Für das Kolloquium können einzelne Ausschnitte bei Bedarf separat gezeigt werden.

3 Vorbereitete digitale Spuren

Damit der Fall forensisch untersucht werden kann, wurden auf den virtuellen Systemen des Verdächtigen gezielt digitale Artefakte vorbereitet. Ein Teil dieser Artefakte wurde anschließend gelöscht oder verborgen, um typische Untersuchungsszenarien der digitalen Forensik nachzustellen. Dadurch können im weiteren Verlauf unter anderem die Wiederherstellung gelöschter Dateien, die Analyse von Metadaten, die Untersuchung stenographischer Inhalte sowie die Auswertung eines Arbeitsspeicherabbildes durchgeführt werden.

Im Folgenden werden die einzelnen Artefakte, ihre Funktion innerhalb des Falls sowie ihr jeweiliger Beweiswert beschrieben.

3.1 Chatverlauf über die Dating-Plattform (parship_chatverlauf.json)

Die Datei parship_chatverlauf.json befindet sich im Verzeichnis /home/simonaman/Dokumente/ und enthält den Chatverlauf zwischen den Benutzerkonten marty_17 und rosanna_15. Sie bildet das zentrale Beweismittel für die Kontaktaufnahme zwischen dem Tatverdächtigen und dem Opfer dar. Der Chat dokumentiert den zeitlichen Verlauf von der ersten Kontaktaufnahme bis zur Vereinbarung eines heimlichen Treffens.

Um ein realistisches Wiederherstellungsszenario zu erzeugen, wurde die Datei gelöscht. Dadurch muss sie im Rahmen der späteren Untersuchung durch Carving des Datenträgers rekonstruiert werden. Erst nach erfolgreicher Wiederherstellung kann der Ablauf der Kontaktaufnahme vollständig nachvollzogen und als Beweismittel bewertet werden.

3.2 Standort-Tracker und Standort-Log (Tracker.py, Locations.log)

Das Skript Tracker.py befindet sich im Verzeichnis /home/simon-aman/Tools/ und simuliert den automatisierten Abruf des Standortes von Rosanna Lorenzo. Hierzu wurde das Skript als

Cron-Job eingerichtet und alle 30 Minuten ausgeführt, wodurch ein realistisches Überwachungsverhalten nachgebildet wird.

Während das Skript selbst auf dem System erhalten bleibt und direkt analysiert werden kann, wurde die erzeugte Log-Datei Locations.log gelöscht. Diese enthält die protokollierten Standortdaten des Mobiltelefons und liefert eines der wichtigsten Beweismittel für den Tatablauf dar. Erst durch ihre Wiederherstellung lässt sich nachweisen, dass sich das Mobiltelefon des Opfers um 22:40 Uhr an der Wohnadresse des Tatverdächtigen befand, bevor die Standortübertragung endet.

3.3 Entführungsplan als Steganografie (plan.jpg)

Im Home-Verzeichnis des Tatverdächtigen befindet sich die Datei plan.jpg. Auf den ersten Blick handelt es sich um eine gewöhnliche Bilddatei. Tatsächlich wurde mit steghide die Textdatei entfuehrungsplan.txt darin verborgen, welche die eigentliche Tatplanung enthält.

Für die Steganografie wurde dasselbe Passwort wie für das Benutzerkonto verwendet (SimonAman98). Dadurch wird ein realistisches Benutzerverhalten simuliert, da Kennwörter in der Praxis häufig mehrfach verwendet werden. Die ursprüngliche Textdatei sowie das unveränderte Ausgangsbild wurden anschließend entfernt, sodass lediglich die präparierte Bilddatei erhalten bleibt.

Der eigentliche Beweiswert ergibt sich erst nach der erfolgreichen Extraktion der versteckten Informationen. Ohne den Verdacht auf steganografisch veränderte Dateien würde der Entführungsplan bei einer oberflächlichen Untersuchung unentdeckt bleiben.

3.4 Standort des Verstecks über EXIF-Metadaten (lager.webp)

Im Download-Verzeichnis des Tatverdächtigen befindet sich die Bilddatei lager.webp. Sie enthält in ihren EXIF-Metadaten GPS-Koordinaten (53°33′37,08″ N, 9°55′37,56″ E), welche auf ein verlassenes Gebäude im Raum Hamburg verweisen und als mögliches Versteck des Opfers dienen.

Die Bilddatei wurde nicht entfernt, da sie ohne die Untersuchung ihrer Metadaten zunächst unauffällig erscheint. Ihr eigentlicher Beweiswert liegt daher nicht im sichtbaren Bildinhalt, sondern in den enthaltenen Standortinformationen, die erst durch Werkzeuge wie exiftool sichtbar werden.

3.5 Bitcoin-Wallet (wallet.dat) – Vorbereitung

Im Verzeichnis /home/simon-aman/Bitcoin/ wurde die Datei wallet.dat abgelegt. Sie enthält eine Bitcoin-Wallet, welche die finanzielle Komponente des Falls repräsentiert und im Zusammenhang mit der späteren Lösegeldforderung steht.

Die Datei wurde gelöscht, sodass sie im Rahmen der Untersuchung zunächst wiederhergestellt werden. Erst die Verknüpfung der Wallet mit der im USB-Container gespeicherten Lösegeldforderung ermöglicht die Rekonstruktion des vollständigen Erpressungsszenarios.

3.6 USB-Container mit weiteren Beweisdateien

Da innerhalb der virtualisierten Laborumgebung kein physischer USB-Stick eingesetzt werden konnte, wurde eine virtuelle Festplatte (USB_Simulation.vhdx) als Ersatz verwendet. Diese wurde zunächst auf einer Kali-Linux-VM mithilfe von cryptsetup (LUKS) verschlüsselt und anschließend mit mehreren Beweisdateien befüllt. Hierzu zählen die Adresse des Verstecks (lageraddresse.txt), die Lösegeldforderung (losegeld.txt) sowie ein Bild des Opfers (rosanna.jpg).

Nach Abschluss der Vorbereitung wurde der verschlüsselte Container ausgehängt und an die Ubuntu-Umgebung des Tatverdächtigen angebunden. Dort wurde der Container erneut geöffnet und vor der Erstellung des Arbeitsspeicherabbildes nicht wieder geschlossen. Dadurch befand sich das Kennwort 12345abc! zum Zeitpunkt der RAM-Sicherung im Arbeitsspeicher. Dieses Vorgehen bildet ein realistisches Szenario einer Live-Forensik nach, bei der ein verschlüsselter Datenträger während des laufenden Betriebs gesichert wird. Der spätere Nachweis des Kennworts im Arbeitsspeicher stellt eine wichtige Voraussetzung für die weitere Untersuchung des Containers dar, da ohne den im Speicher vorhandenen Schlüssel beziehungsweise das Kennwort eine Entschlüsselung des verschlüsselten Containers erheblich erschwert wäre.

4 Asservate und Chain of Custody

Im Rahmen der Vorbereitung des Falls wurden mehrere digitale Beweismittel erstellt und für die spätere Untersuchung gesichert. Diese Asservate bilden die Grundlage der forensischen Analyse und ermöglichen die Rekonstruktion des fiktiven Tatablaufs. Um die Integrität der Beweismittel sicherzustellen, wurden alle Datenträgerabbilder unmittelbar nach ihrer Erstellung mittels Hashwerten überprüft und dokumentiert. Dadurch kann nachgewiesen werden, dass während der Untersuchung keine Veränderungen an den Ausgangsdaten vorgenommen wurden.

4.1 Übersicht der Asservate

Die nachfolgende Tabelle gibt einen Überblick über die im Rahmen der Untersuchung verwendeten Asservate sowie deren Herkunft, Sicherungsmethode und späteren Untersuchungszweck.

Nr.	Asservat	Sicherungsmethode	Quelle	Format	Forensischer Zweck
A-1	System-Datenträger (Ubuntu „simon-aman“)	FTK Imager (Image File)	Virtuelle Festplatte (VHDX)	Raw (dd), fragmentiert	Analyse des Dateisystems sowie Wiederherstellung gelöschter Dateien
A-2	USB-Container (Simulation)	Kein separates forensisches Image	USB_Simulation.vhdx	Virtuelle Festplatte (VHDX)	Untersuchung des verschlüsselten Containers nach erfolgreicher Entschlüsselung
A-3	Arbeitsspeicherabbild	AVML	Laufendes UbuntuSystem	LiME, komprimiert	Analyse flüchtiger Daten, insbesondere Passwort- und Prozessinformationen

Die drei Asservate ergänzen sich gegenseitig und ermöglichen erst in ihrer Gesamtheit eine vollständige Rekonstruktion des Falls. Während das System-Image die Untersuchung des Dateisystems sowie die Wiederherstellung gelöschter Dateien ermöglicht, enthält der Arbeitsspeicher flüchtige Informationen wie Passwörter oder laufende Prozesse. Der verschlüsselte USB-Container stellt zusätzliche Beweismittel bereit, deren Auswertung erst nach erfolgreicher Entschlüsselung möglich ist.

4.2 Verwendete Untersuchungsumgebung

Für die Erstellung der Beweismittel sowie deren Sicherung kamen mehrere virtuelle Maschinen mit unterschiedlichen Aufgaben zum Einsatz.

Virtuelle Maschine	Betriebssystem	Aufgabe
Kali-Linux-VM	Kali Linux	Erstellung und Verschlüsselung des USB-Containers mittels LUKS (cryptsetup)
Ubuntu-VM „simon-aman“	Ubuntu	Simulation des Systems des Tatverdächtigen sowie Ablage der digitalen Artefakte
Windows-Analyse-VM	Windows 11	Erstellung der forensischen Images mit FTK Imager sowie Hashwertbildung

Zunächst wurde der USB-Container auf der Kali-Linux-VM erstellt, verschlüsselt und mit den vorbereiteten Beweismitteln befüllt. Anschließend wurde die virtuelle Festplatte an die UbuntuVM des Tatverdächtigen angebunden. Die Erstellung der forensischen Datenträgerabbilder erfolgte anschließend auf einer separaten Windows-Analyse-VM, auf der die jeweiligen VHDX-Dateien mit FTK Imager eingelesen und als Raw-(dd)-Images exportiert wurden.

4.3 Chain of Custody

Ein wesentliches Ziel der digitalen Forensik besteht darin, die Integrität und Nachvollziehbarkeit digitaler Beweismittel während des gesamten Untersuchungsprozesses sicherzustellen. Aus diesem Grund wurden sämtliche Asservate unmittelbar nach ihrer Sicherung durch kryptografische Hashwerte verifiziert und dokumentiert. Die dabei erzeugten Prüfsummen dienen als Referenzwerte und ermöglichen den Nachweis, dass die Beweismittel während der weiteren Untersuchung unverändert geblieben sind.

Da die Untersuchung vollständig innerhalb einer virtualisierten Laborumgebung durchgeführt wurde, kam kein physischer Hardware-Write-Blocker zum Einsatz. Stattdessen erfolgte die Erstellung der Images auf Grundlage der virtuellen Festplatten, welche ausschließlich lesend verarbeitet wurden. Die eigentliche Integritätsprüfung erfolgte anschließend durch den Vergleich der berechneten Hashwerte mit den dokumentierten Referenzwerten. Eine detaillierte Darstellung der verwendeten Hashverfahren und der ermittelten Prüfsummen erfolgt in Kapitel 8.

5 Erkennung – Warum der Verdacht auf Simon Aman fiel

Die 15-jährige Rosanna Lorenzo verbringt den Abend des 6. März 2026 mit ihren Freundinnen in einem Club. Gegen 22:30 Uhr verlässt sie die Gruppe und macht sich allein auf den Heimweg. Sie kommt jedoch nie zu Hause an. Als sie am nächsten Morgen weiterhin verschwunden ist, alarmieren ihre Eltern die Polizei.

Bei der Durchsicht von Rosannas Tablet stellen die Eltern fest, dass sie seit mehreren Wochen auf einer Dating-Plattform aktiv war und dort intensiven Kontakt zu einem Nutzer mit dem Namen „marty_17" hatte. Die letzte Nachricht dieses Kontakts kündigt an, Rosanna noch am selben Abend vom Club abzuholen. Kurz darauf erhalten die Eltern zudem eine anonyme Nachricht, die eine Geldforderung in Bitcoin enthält und den Hinweis, dass sich ihre Tochter in der Gewalt des Absenders befinde. Die Polizei geht daraufhin von einer Entführung aus.

Die Auswertung einer städtischen Überwachungskamera in der Nähe des Clubs zeigt Rosanna um 22:30 Uhr, wie sie in ein dunkles Fahrzeug einsteigt, einen Audi A2. Das Kennzeichen ist auf der Aufnahme nicht lesbar.

Auf Antrag der Ermittler wird die IP-Adresse hinter dem Profil marty_17 zurückverfolgt, sie führt zu Simon Aman, 28 Jahre alt, wohnhaft nur wenige Kilometer vom Tatort entfernt. Ergänzend fällt ins Gewicht, dass gegen Aman bereits in der Vergangenheit polizeilich wegen eines ähnlich gelagerten Vorfalls mit einer minderjährigen Person ermittelt wurde, und dass er einen Audi A2 besitzt, der dem im Kamerabild gezeigten Fahrzeug entspricht. In der Zusammenschau dieser Hinweise entsteht ein hinreichender Anfangsverdacht, sodass eine Hausdurchsuchung sowie die Sicherstellung der IT-Systeme von Simon Aman noch am selben Tag beantragt und richterlich angeordnet werden.

Bei Eintreffen der Beamten ist Simon Aman nicht in seiner Wohnung anwesend. Dort werden jedoch mehrere digitale Geräte sichergestellt, deren forensische Untersuchung Gegenstand der folgenden Kapitel ist.

6 Initial Response (Erstzugriff am Tatort)

Bei der Sicherstellung am 7. März 2026 wird das Notebook von Simon Aman im eingeschalteten und ungesperrten Zustand aufgefunden. Eine erste vorsichtige Sichtung der geöffneten Sitzung zeigt, dass kurz zuvor bereits Befehle mit administrativen Rechten (sudo) ausgeführt wurden. Für die ersten Sicherungsmaßnahmen ist daher zunächst keine erneute Authentifizierung erforderlich, da die Sitzung bereits entsperrt ist.

Da flüchtige Daten wie der Arbeitsspeicher grundsätzlich vor nichtflüchtigen Datenträgern gesichert werden sollten, erfolgt zunächst die Sicherung des Arbeitsspeichers mittels AVML.

Erst anschließend werden die virtuellen Datenträger für die Erstellung forensischer Images gesichert. Diese Reihenfolge ist entscheidend, da sich Informationen wie Passwörter oder Entschlüsselungsschlüssel ausschließlich im flüchtigen Speicher befinden können. Im vorliegenden Fall ermöglicht dies später den Nachweis des Kennworts für den verschlüsselten USB-Container.

Der Zustand des Systems vor Beginn der Sicherungsmaßnahmen wird dokumentiert. Anschließend werden die einzelnen Schritte der Beweissicherung, beginnend mit der RAMSicherung und anschließend der Erstellung der Datenträgerabbilder, im Sicherungsprotokoll festgehalten.



Abbildung 1: Schematische Darstellung des Erstzugriffs auf das Notebook (KI-generiert mit Google Gemini).

7 Methodik und eingesetzte Werkzeuge

Dieses Kapitel benennt die für die Sicherung und Auswertung eingesetzten Werkzeuge und beschreibt den methodischen Rahmen der Untersuchung. Dabei wird durchgehend zwischen einem Werkzeug (einem eigenständigen Programm, z. B. The Sleuth Kit, Volatility 3 oder cryptsetup) und einem einzelnen Befehl bzw. Teilprogramm eines Werkzeugs (z. B. mmls, fls und icat als Befehle des The Sleuth Kit oder linux.bash als Plugin von Volatility 3) unterschieden. Die konkrete Anwendung der Werkzeuge auf den Fall wird in Kapitel 9 dokumentiert; eine vollständige Aufstellung aller Werkzeuge und der jeweils tatsächlich verwendeten Befehle findet sich in der begleitenden Datei Fall4321_Forensic.xlsx.

7.1 Eingesetzte Software

Die nachfolgende Tabelle fasst die eingesetzten Werkzeuge und ihren jeweiligen Einsatzzweck zusammen. Fallspezifische Werkzeuge (steghide, cryptsetup, John the Ripper, hashcat) wurden erst dort über den Paketmanager installiert, wo ein Befund ihren Einsatz rechtfertigte.

Werkzeug	Einsatzzweck
FTK Imager	Erstellung der Datenträger-Images (Raw/dd) sowie deren Verifikation über MD5-/SHA1-Hashwerte.
AVML (Microsoft)	Erstellung des Arbeitsspeicher-Abbildes des laufenden Ubuntu-Systems (LiME-Format, komprimiert).
PowerShell Get-FileHash / sha256sum	Neuberechnung der Prüfsummen zur Integritätsprüfung der Asservate.
The Sleuth Kit	Dateisystemforensik am System-Image: Partitionsanalyse (mmls), Dateisystembestätigung (fsstat), rekursive Auflistung und Inode-Ermittlung (fls), verlustfreie Extraktion vorhandener Dateien (icat) sowie Nachweis vollständig entfernter Verzeichniseinträge (fls -rd).
carve_deleted.py (PythonEigenentwicklung)	Inhaltsbasierte Wiederherstellung der gelöschten Dateien parship_chatverlauf.json und Locations.log durch Carving des unallozierten Speicherbereichs, da keine Verzeichniseinträge mehr vorhanden waren.
Volatility 3	Arbeitsspeicherforensik mit dem Plugin linux.bash zur Rekonstruktion der Befehlshistorie und zum Nachweis, dass der USB-Container zum Sicherungszeitpunkt geöffnet war.
strings (GNU Binutils)	Extraktion aller druckbaren Zeichenketten aus dem RAM-Abbild zu einer Kandidaten-Wortliste (ram_wordlist.txt); es erfolgt keine gezielte Suche nach einem bereits bekannten Kennwort.
John the Ripper	Rückgewinnung des Kontopassworts durch Knacken des /etc/shadowHashes (SHA-512) mit der aus dem RAM erzeugten Wortliste.
OpenSSL (openssl passwd)	Unabhängiger Nachvollzug, dass das ermittelte Kontopasswort dem gespeicherten /etc/shadow-Hash entspricht.
cryptsetup / LUKS	Identifikation des USB-Containers (luksDump), Validierung der Passphrase gegen den LUKS-Header (--test-passphrase) und Entschlüsselung des Containers (luksOpen). hashcat stand als alternatives, GPU-gestütztes Verfahren zur Validierung gegen den gesicherten LUKS-Header zur Verfügung.
steghide	Nachweis und Extraktion der in plan.jpg verborgenen Textdatei mit dem zuvor rückgewonnenen Kontopasswort.
exiftool	Auslesen der GPS-Koordinaten aus den EXIF-Metadaten von lager.webp.

7.2 Werkzeuge im Detail

Für die Sicherung und Auswertung der Beweismittel kamen Werkzeuge zum Einsatz, die jeweils unterschiedliche Aufgaben innerhalb der Untersuchung übernehmen.

FTK Imager diente der Erstellung der forensischen Datenträgerabbilder aus den virtuellen Festplatten. Es ermöglicht zugleich die unmittelbare Verifikation der erzeugten Images über den Vergleich der berechneten MD5- und SHA1-Hashwerte, sodass bereits während der Image-Erstellung nachgewiesen werden kann, dass die Kopie dem Ausgangsmedium entspricht.

Zur Sicherung des Arbeitsspeichers wurde AVML (Acquire Volatile Memory for Linux) verwendet. Das Werkzeug eignet sich für Linux-Systeme, da es ohne aufwendige Installation ausgeführt werden kann und den laufenden Systemzustand möglichst wenig beeinflusst. Das Speicherabbild wurde komprimiert im LiME-Format abgelegt.

Die dateisystemnahe Erschließung des System-Images stützte sich auf The Sleuth Kit, dessen Befehle mmls, fsstat, fls und icat die Partitions- und Dateisystemanalyse, die Adressierung über InodeNummern und die verlustfreie Extraktion vorhandener Dateien ermöglichen. Da die bewusst gelöschten Dateien keine Verzeichniseinträge mehr besaßen und damit einer namensbasierten Wiederherstellung nicht zugänglich waren, wurde für deren inhaltsbasierte Rekonstruktion ein eigens erstelltes Python-Skript (carve_deleted.py) eingesetzt. Dieses Skript ist von The Sleuth Kit zu unterscheiden: Es führt kein dateisystembasiertes Undelete durch, sondern durchsucht den unallozierten Speicherbereich anhand von Strukturmerkmalen, die aus den vorherigen Befunden abgeleitet wurden.

Die inhaltliche Auswertung erfolgte mit spezialisierten Werkzeugen: Volatility 3 (Plugin linux.bash) für die Rekonstruktion der Befehlshistorie aus dem Arbeitsspeicher, exiftool für die Auswertung von EXIF/GPS-Metadaten und steghide für die Extraktion steganografisch eingebetteter Inhalte. Für die methodisch saubere Rückgewinnung von Zugangsdaten wurden strings (Erzeugung der KandidatenWortliste), John the Ripper und OpenSSL (Kontopasswort gegen den /etc/shadow-Hash) sowie cryptsetup (Container-Passphrase gegen den LUKS-Header) verwendet. Die konkrete Anwendung dieser Werkzeuge wird in Kapitel 9 beschrieben.

7.3 Untersuchungsablauf

Die Untersuchung folgte einem streng artefaktgestützten Ablauf, bei dem sich jeder Schritt aus den Befunden des vorangegangenen ergibt. Zunächst wurde die Integrität der drei Asservate durch einen Abgleich neu berechneter Prüfsummen mit den dokumentierten Referenzwerten bestätigt (Kapitel 8), da ohne Integritätsnachweis keine der nachfolgenden Erkenntnisse Beweiswert besitzt.

Anschließend wurde das System-Image erschlossen: Aus der Partitionstabelle wurde der Offset der Linux-Partition abgeleitet, das ext4-Dateisystem bestätigt und über /etc/passwd der einzige reguläre Benutzer (simon-aman) bestimmt. Auf dieser Grundlage wurden die Benutzerartefakte ausgewertet – vor allem der Shell-Verlauf, die Cron-Konfiguration und die zuletzt verwendeten Dateien –, die als Leitartefakte sämtliche weiteren Untersuchungsstränge begründen.

Aus den in diesen Artefakten benannten, jedoch nicht mehr vorhandenen Dateien ergab sich die Wiederherstellung der gelöschten Dateien durch Carving. Erst danach folgte die inhaltliche Auswertung der einzelnen Spuren in der Reihenfolge, in der sie sich aus den Befunden ergaben: Chatverlauf, Standortprotokoll, EXIF-Metadaten und Bitcoin-Wallet.

Für die beiden kennwortgeschützten Vorgänge – die Steganografie und der verschlüsselte Container – wurde das jeweilige Kennwort nicht vorausgesetzt. Da der Shell-Verlauf und die Arbeitsspeicheranalyse belegen, dass der Container zum Sicherungszeitpunkt geöffnet und das Benutzerkonto angemeldet war, mussten die zugehörigen Zeichenketten im Arbeitsspeicher vorliegen. Daher wurde zunächst die Arbeitsspeicheranalyse durchgeführt und aus dem RAM eine Kandidaten-Wortliste erzeugt. Das Kontopasswort wurde durch Knacken des /etc/shadow-Hashes hergeleitet und anschließend auf die Steganografie angewendet; die LUKS-Passphrase wurde nach demselben Prinzip gegen den LUKS-Header validiert und der Container erst danach entschlüsselt. Auf Grundlage aller gewonnenen Erkenntnisse wurden abschließend die Zeitachse und die Beweiskette zusammengeführt.

7.4 Wahl des Image-Formats (Raw/dd)

Für die Sicherung der Datenträger wurde das Raw-(dd)-Format verwendet. Dieses Format erzeugt eine sektorweise 1:1-Kopie des ursprünglichen Datenträgers ohne Kompression oder zusätzliche

Containerinformationen, sodass sich die Images mit einer Vielzahl forensischer Werkzeuge ohne vorherige Konvertierung analysieren lassen.
Alternativ hätte das E01-Format verwendet werden können, das zusätzlich Metadaten und Prüfsummen innerhalb der Image-Datei speichert und aufgrund integrierter Kompression in der Regel weniger Speicherplatz benötigt. Da die Integrität der Beweismittel in dieser Projektarbeit separat durch Hashwertbildung dokumentiert wurde und der Schwerpunkt auf einer möglichst einfachen Weiterverarbeitung lag, wurde das Raw-(dd)-Format gewählt.

8 Beweissicherung – Images und Hashwerte

In diesem Kapitel wird dokumentiert, wie die drei Asservate forensisch gesichert wurden und welche Prüfsummen dabei erzeugt und verifiziert wurden.

8.1 Integritätsprüfung mittels Hashwerten

Vor jeder inhaltlichen Auswertung wurde die Integrität der Asservate überprüft, um die Beweiskraft der nachfolgenden Ergebnisse sicherzustellen. Für das System- und das Arbeitsspeicherabbild wurden die kryptografischen Prüfsummen neu berechnet und mit den bei der Sicherung dokumentierten Referenzwerten verglichen. Für das Systemabbild stimmten die berechneten Werte (MD5 7b011b20b64387b32ac41fac5c2e0967, SHA1 06812d976377cba0c7ab7b37e70a2b5bd1f6bb50) mit den Referenzwerten überein; das Abbild ist damit nachweislich unverändert. Für das Arbeitsspeicherabbild ergab die erneute SHA256-Berechnung den Wert 00A717F07F3ACDD1CADF8A1D8BBA884196C4E7B3F13B57D044BEC2B16A1759B3, der ebenfalls mit dem Referenzwert übereinstimmt. Für den verschlüsselten USB-Container lag bereits eine dokumentierte Integritätsprüfung vor: Der berechnete SHA1-Hash (1C8AF3C35BF095E8E0792AAC66BEB90F457557DF) stimmt mit dem Referenzhash überein; die Integrität dieses Beweisdatenträgers ist damit bestätigt.

Asservat	Hashwert-Berechnung	Hashwert-Verifikation
SimonA man (SystemImage)	MD5 7b011b20b64387b32ac41fac5c2e0967 / SHA1 06812d976377cba0c7ab7b37e70a2b5bd1f6bb50	MD5 7b011b20b64387b32ac41fac5c2e0967 / SHA1 06812d976377cba0c7ab7b37e70a2b5bd1f6bb50
RAM_DU MP	SHA256 00A717F07F3ACDD1CADF8A1D8BBA884196C4E7 B3F13B57D044BEC2B16A1759B3	SHA256 00A717F07F3ACDD1CADF8A1D8BBA884196C4E7 B3F13B57D044BEC2B16A1759B3
USB_Simulation	SHA1 1C8AF3C35BF095E8E0792AAC66BEB90F457557DF	SHA1 1C8AF3C35BF095E8E0792AAC66BEB90F457557DF

8.2 Übersicht der erzeugten Images

Image	Quelle	Format
Ubuntu_image.dd.001 ff.	VHDX der Ubuntu-VM „simon-aman“	Raw (dd), fragmentiert
ram_dump.lime.compressed	Arbeitsspeicher des laufenden UbuntuSystems	LiME, komprimiert

8.3 Übersicht der Beweismittel

Die folgende Tabelle stellt die gesicherten Beweismittel zusammen. Dabei ist zwischen den durch Carving wiederhergestellten Dateien (parship_chatverlauf.json und Locations.log) und der noch vorhandenen, per icat gesicherten Datei wallet.dat zu unterscheiden.

Beweismittel	Herkunft
Ubuntu_image.dd.001 ff. inkl. Hashwerte	System-Datenträger des Tatverdächtigen
USB_Simulation.vhdx inkl. Hashwerte	Simulierter USB-Container (LUKS)

ram_dump.lime.compressed inkl. Hashwerte	Arbeitsspeicher des laufenden Systems
parship_chatverlauf.json (24 Nachrichten), Locations.log (30 Einträge)	Durch Carving des unallozierten Speicherbereichs rekonstruierte, zuvor gelöschte Dateien
wallet.dat	Auf dem System vorhandene Datei (/home/simonaman/bitcoin), per icat aus dem System-Image gesichert

9 Auswertung

In diesem Kapitel werden die Ergebnisse der forensischen Untersuchung dokumentiert. Grundlage der Analyse bilden die in Kapitel 3 beschriebenen Artefakte sowie die in Kapitel 8 gesicherten Datenträgerabbilder. Die Unterkapitel folgen dem tatsächlichen Untersuchungsablauf; jeder Schritt baut auf den Ergebnissen des vorherigen auf und wird durch die verwendeten Werkzeuge, Befehle und Screenshots belegt.

9.1 Methodik und Untersuchungsansatz

Die Untersuchung folgte den Phasen Identifikation, Sicherung, Erfassung, Untersuchung, Analyse und Berichterstattung. Sämtliche Zugriffe erfolgten ausschließlich lesend auf forensischen Kopien; die Originaldaten wurden zu keinem Zeitpunkt verändert.

Zentraler Grundsatz war das artefaktgestützte Vorgehen: Es wurde nicht gezielt nach vermuteten Dateinamen, Kennwörtern oder Inhalten gesucht, sondern das System systematisch anhand seiner Betriebssystem- und Benutzerartefakte erschlossen. Wo Zugangsdaten benötigt wurden, wurden aus dem Speicherabbild gewonnene Kandidaten gegen unabhängige Prüfinstanzen validiert, statt sie vorauszusetzen. Verwendet wurde eine schreibgeschützte Analyseumgebung auf Basis von Ubuntu unter dem Windows-Subsystem für Linux (WSL2) mit The Sleuth Kit, exiftool, steghide, cryptsetup und Volatility 3.

9.2 Systemidentifikation und Dateisystemstruktur

Im ersten Untersuchungsschritt wurde die Struktur des Systemabbildes erschlossen, da ohne Kenntnis von Partitionierung und Dateisystem keine gezielte Auswertung möglich ist. Die Partitionstabelle wurde mit mmls gelesen:

```
mmls simon.dd
```

Die Ausgabe zeigte eine GPT-Struktur mit einer EFI-Partition sowie einer Linux-Systempartition. Der Startsektor der Linux-Partition (Sektor 2203648) wurde unmittelbar aus dieser Ausgabe abgeleitet; alle weiteren Dateisystemzugriffe erfolgten über diesen Offset. Das Dateisystem wurde anschließend mit fsstat bestätigt:

```
fsstat -o 2203648 simon.dd
```

Das Dateisystem wurde als ext4 identifiziert, zuletzt unter dem Wurzelverzeichnis („/“) eingehängt und ordnungsgemäß ausgehängt, mit vorhandenem Journal.

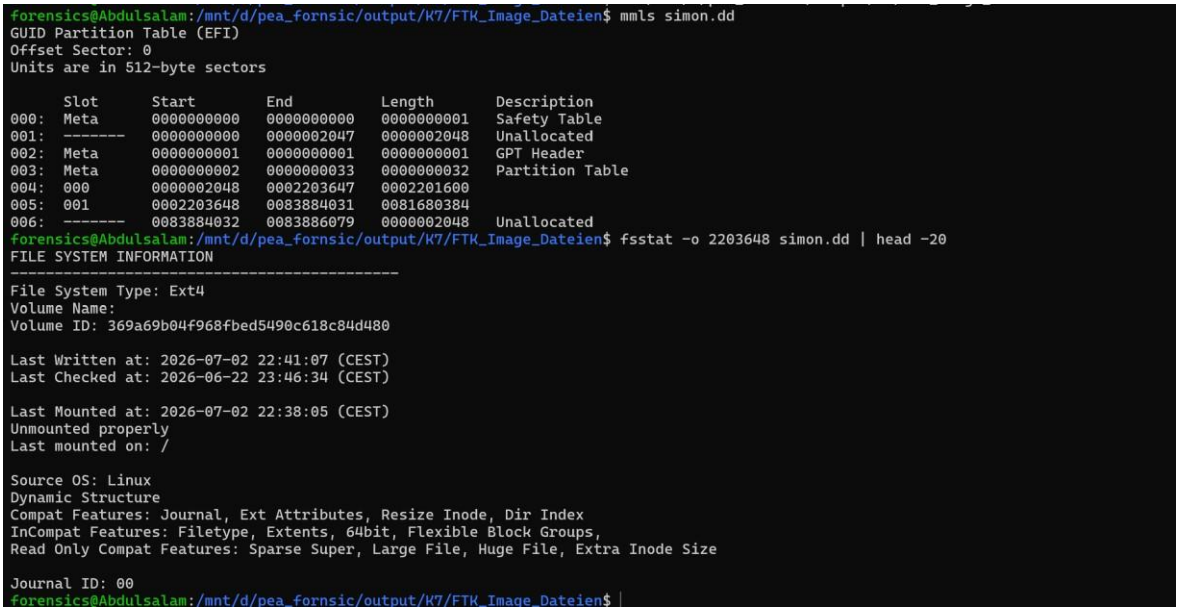


Abbildung 2: Partitionstabelle und Dateisystem (mmls, fsstat) – Linux-Partition ab Sektor 2203648, ext4

Zur Bestimmung der auf dem System aktiven Personen wurde /etc/passwd ausgewertet. Die InodeNummer wurde dabei dynamisch über das Pfadmuster ermittelt und die Datei anschließend mit icat ausgelesen:

```
icat -o 2203648 simon.dd 133857 | awk -F: '$3>=1000 && $3<65534 {print $1,$6,$7}'
```

Die Auswertung wies genau ein reguläres Benutzerkonto mit einer Benutzerkennung ab 1000 aus: den Benutzer simon-aman (UID 1000, Home-Verzeichnis /home/simon-aman, Anmelde-Shell /bin/bash). Der Hostname des Systems lautet ebenfalls simon-aman. Die weitere Untersuchung konzentrierte sich daher auf dessen Benutzerverzeichnis.



Abbildung 3: Ermittlung der Inode-Nummer und Auslesen von /etc/passwd (simon-aman, UID 1000)

Ergänzend wurde /etc/shadow gesichert. Das Konto simon-aman besitzt ein Passwort im SHA-512Verfahren (Kennung 6). Dieser Hashwert wird in Abschnitt 9.9 als unabhängige Prüfinstanz für die Rückgewinnung des Kontopassworts herangezogen.

9.3 Untersuchung des Benutzerprofils

Zur Erschließung der Benutzeraktivität wurden die typischen Profil- und Betriebssystemartefakte des Kontos simon-aman ausgewertet. Diese Artefakte liefern die Anhaltspunkte, aus denen sich alle weiteren Untersuchungsschritte ergeben.

9.3.1 Auswertung des Shell-Verlaufs (.bash_history)

Die Datei .bash_history protokolliert die zuvor eingegebenen Shell-Befehle und stellte sich als aussagekräftigstes Artefakt der Untersuchung heraus. Sie wurde über ihre Inode-Nummer extrahiert und lesbar dargestellt:

```
icat -o 2203648 simon.dd 1835972 | cat -v
```

Ihr Inhalt dokumentiert in zeitlicher Abfolge eine Reihe auffälliger Handlungen des Benutzers. Zunächst wird steghide installiert und verwendet, um eine Textdatei in eine Bilddatei (plan.jpg) einzubetten.

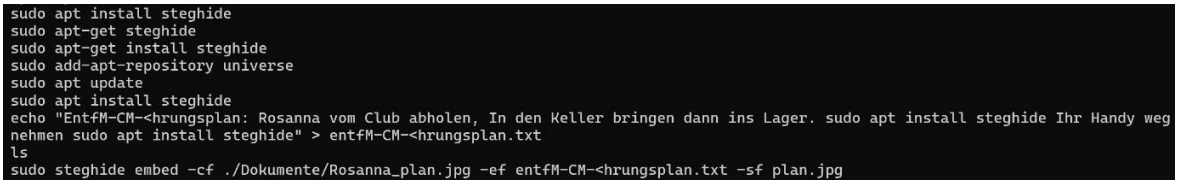


Abbildung 4: Shell-Historie – Einbetten der Textdatei mit steghide (steghide embed … -sf plan.jpg)

Anschließend wird ein Python-Skript tracker.py erstellt, mehrfach ausgeführt und ein zugehöriger Cron-Eintrag eingerichtet.



Abbildung 5: Shell-Historie – Erstellung, Ausführung und Cron-Einrichtung von tracker.py

Es folgt eine nachträgliche Manipulation der GPS-Metadaten einer Bilddatei mittels exiftool.



Abbildung 6: Shell-Historie – Manipulation der GPS-Metadaten mit exiftool (lager.webp) Zudem

wird eine Datei wallet.dat mit Bezug zu einer Kryptowährung erzeugt.

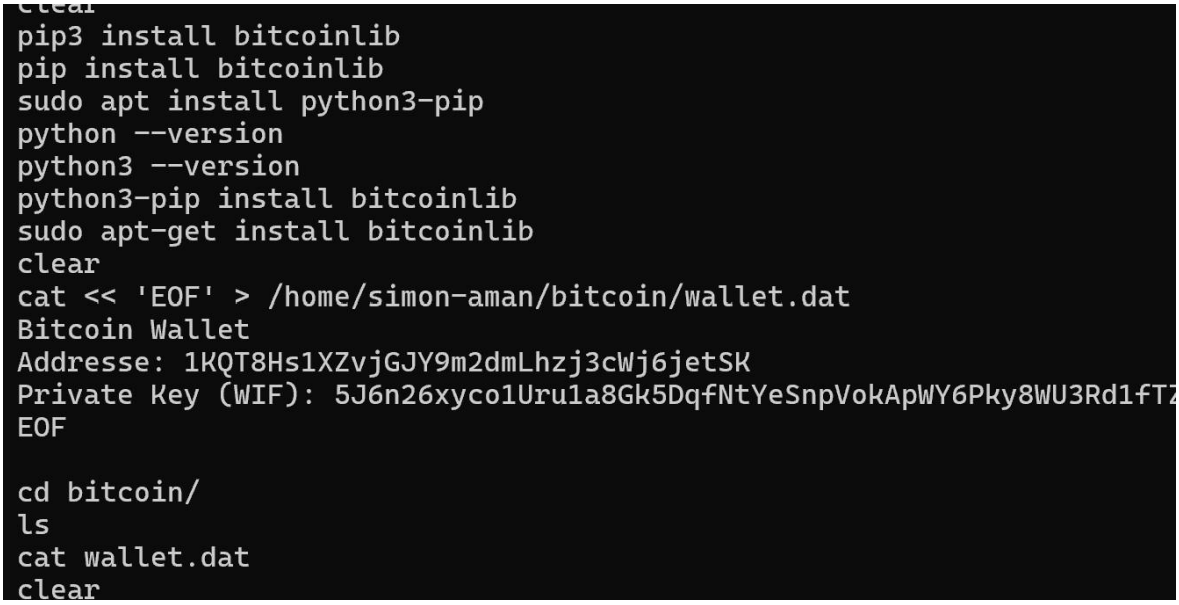


Abbildung 7: Shell-Historie – Erzeugung der Datei wallet.dat

Ein verschlüsselter Datenträger wird über cryptsetup luksOpen geöffnet und eingehängt.



Abbildung 8: Shell-Historie – Öffnen des USB-Containers mit cryptsetup luksOpen

Anschließend werden gezielt mehrere Dateien gelöscht, darunter parship_chatverlauf.json, locations.log sowie das unveränderte Ausgangsbild lager.webp_original.

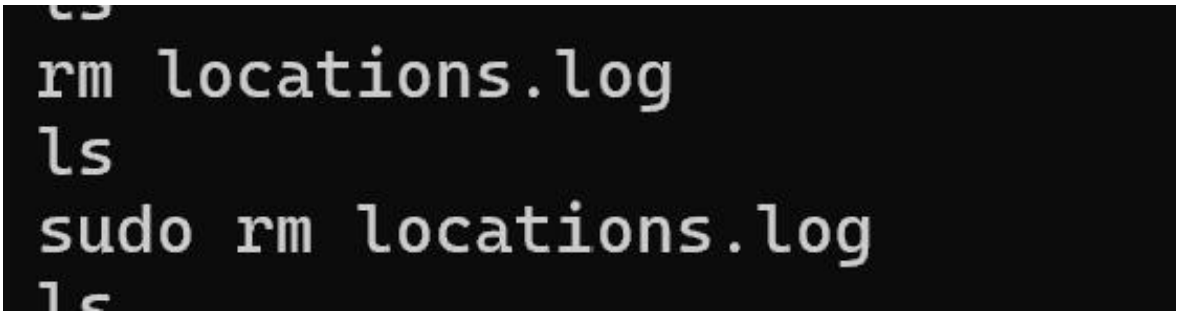


Abbildung 9: Shell-Historie – Löschen der Datei locations.log

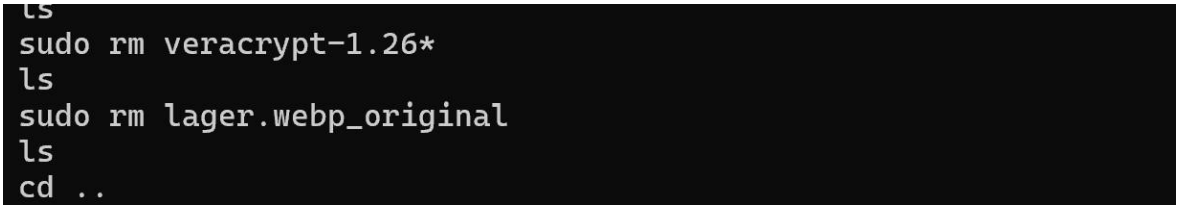


Abbildung 10: Shell-Historie – Entfernen des VeraCrypt-Installers und von lager.webp_original

Abschließend wird der Arbeitsspeicher mit AVML gesichert, der Datenträger mittels dd dupliziert und das System heruntergefahren.



Abbildung 11: Shell-Historie – abschließende Sicherungsschritte und System-Shutdown

Der Shell-Verlauf begründet damit sämtliche nachfolgenden Untersuchungsstränge: die Suche nach verborgenen Daten in einer Bilddatei (9.10), die Auswertung des Standort-Skripts (9.6), die Prüfung von Bildmetadaten (9.7), die Untersuchung der Wallet-Datei (9.8), die Existenz eines verschlüsselten Containers (9.9, 9.11) sowie die Wiederherstellung der gelöschten Dateien (9.4).

9.3.2 Auswertung geplanter Aufgaben (Cron)

Da der Shell-Verlauf auf die Einrichtung einer geplanten Aufgabe hinwies, wurde die benutzerspezifische Cron-Konfiguration geprüft:

```
icat -o 2203648 simon.dd 1574099 | grep -vE '^#|^$'
```

Die Crontab enthält genau einen aktiven Eintrag, der das Skript tracker.py alle 30 Minuten ausführt (*/30 * * * *). Dies belegt einen automatisierten, wiederkehrenden Vorgang und lenkt die weitere Untersuchung auf das Skript und dessen Ausgabe.



Abbildung 12: Cron-Eintrag für tracker.py (halbstündliche Ausführung)

9.3.3 Auswertung zuletzt verwendeter Dateien (recently-used.xbel)

Zur unabhängigen Absicherung der bisherigen Befunde wurde das Artefakt der zuletzt verwendeten Dateien ausgewertet:

```
icat -o 2203648 simon.dd 1837995 | grep -oE 'href="[^"]*"'
```

Die Datei recently-used.xbel verweist unter anderem auf Dokumente/parship_chatverlauf.json, Dokumente/Rosanna_plan.jpg sowie Downloads/lager.webp. Diese Referenzen bestätigen unabhängig vom Shell-Verlauf die frühere Existenz dieser Dateien und benennen konkrete Speicherorte, die im folgenden Schritt mit dem tatsächlichen Dateibestand abzugleichen sind.



Abbildung 13: Referenzen in recently-used.xbel

9.4 Wiederherstellung gelöschter Dateien

Die Befunde aus 9.3 nennen mehrere Dateien, deren tatsächlicher Bestand nun zu überprüfen war. Hierzu wurde der aktuelle Dateibestand des Benutzerverzeichnisses mit den referenzierten Pfaden abgeglichen. Das Verzeichnis Dokumente, in dem laut recently-used.xbel die Datei parship_chatverlauf.json lag, war leer. Das Verzeichnis tools enthielt zwar das Skript tracker.py, jedoch nicht die von diesem erzeugte Protokolldatei Locations.log – obwohl die geplante Aufgabe diese über Tage regelmäßig hätte erzeugen müssen. Beide Beobachtungen bestätigen die im Shell-Verlauf dokumentierten Löschungen.

Eine Analyse der Verzeichniseinträge auf Dateisystemebene ergab, dass für beide Dateien keine gelöschten Einträge mehr auffindbar waren:

```
fls -o 2203648 -rd simon.dd | grep -iE 'parship|locations'
```

Der Befehl lieferte keine Treffer. Die Namensreferenzen wurden vom ext4-Dateisystem vollständig entfernt, sodass eine namensbasierte Wiederherstellung (Undelete) ausgeschlossen war.

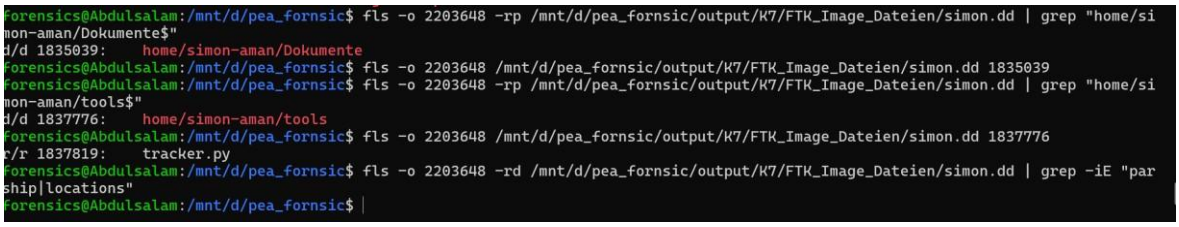


Abbildung 14: Prüfung auf gelöschte Verzeichniseinträge (fls -rd) – kein Treffer

Die Wiederherstellung erfolgte daher inhaltsbasiert durch ein gezieltes Carving des unallozierten Speicherbereichs mit einem eigens erstellten Python-Skript. Die verwendeten Suchmerkmale wurden aus den vorherigen Befunden abgeleitet und setzten keine Kenntnis der Inhalte voraus: für die ChatDatei der aus dem Dateinamen bekannte Plattformname innerhalb einer gültigen JSON-Struktur, für das Protokoll das aus dem noch lesbaren Skript tracker.py bekannte Zeilenformat.

```
python3 carve_deleted.py simon.dd recovered_carve
```

Wiederhergestellt wurden ausschließlich die beiden bewusst gelöschten Dateien: der Chatverlauf parship_chatverlauf.json als gültige JSON-Datei mit 24 Nachrichten sowie das Standortprotokoll Locations.log mit 30 Einträgen. Die Bitcoin-Wallet wallet.dat war hingegen nicht gelöscht und wird daher nicht durch Carving, sondern als vorhandene Datei in Abschnitt 9.8 behandelt. Die inhaltliche Auswertung der wiederhergestellten Dateien erfolgt in den Abschnitten 9.5 und 9.6.

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ python3 /mnt/d/pea_fornsic/output/K7/carve_deleted.py /mnt/d/pea_fornsic/output/K7/FTK_Ima
ge_Dateien/simon.dd /mnt/d/pea_fornsic/output/K7/recovered_carve
PS D:\pea_fornsic> wsl
forensics@Abdulsalam:/mnt/d/pea_fornsic$ ls -la /mnt/d/pea_fornsic/output/K7/recovered_carve
total 12
drwxrwxrwx 1 forensics forensics  512 Jul  6 16:10 .
drwxrwxrwx 1 forensics forensics  512 Jul  6 20:09 ..
-rwxrwxrwx 1 forensics forensics 2388 Jul  6 16:10 Locations.log
-rwxrwxrwx 1 forensics forensics 6843 Jul  6 16:07 parship_chatverlauf.json
forensics@Abdulsalam:/mnt/d/pea_fornsic$
```

Abbildung 15: Wiederhergestellte Dateien im Verzeichnis recovered_carve (parship_chatverlauf.json, Locations.log)

9.5 Chat-Verlauf (parship_chatverlauf.json)

Nach der Wiederherstellung wurde der Chatverlauf strukturiert und lesbar aufbereitet:

```
python3 -m json.tool parship_chatverlauf.json | less
```

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ python3 -m json.tool /mnt/d/pea_fornsic/output/K7/recovered_carve/parship_chatverlauf.json
 | less
```

Abbildung 16: Aufbereitung von parship_chatverlauf.json (python3 -m json.tool)

Die Datei enthält den Nachrichtenverlauf einer Dating-Plattform zwischen den Konten marty_17 und rosanna_15. Der Verlauf beginnt am 28. Februar 2026 und entwickelt sich über mehrere Tage; insgesamt umfasst er 24 Nachrichten. Auffällig ist das wiederholte Drängen auf ein Treffen, das ausdrücklich vor den Begleiterinnen des Gegenübers verheimlicht werden soll.


```
"messages": [
    {
        "id": "msg_001",
        "from": "marty_17",
        "to": "rosanna_15",
        "timestamp": "2026-02-28T18:20:00+02:00",
        "text": "Hey Rosi \ud83d\ude0a Dein Profil sieht voll sympathisch aus. Ich bin Marty. Kommst du auch aus Hamburg?",
        "type": "text"
    },
    {
        "id": "msg_002",
        "from": "rosanna_15",
        "to": "marty_17",
        "timestamp": "2026-02-28T19:45:00+02:00",
        "text": "Hi \ud83d\ude0a Ja wohne in Barmbek. Und du?",
        "type": "text"
```


Abbildung 17: Chatverlauf – Beginn der Kontaktaufnahme (28.02.2026)

Für den 6. März 2026 wird ein konkretes Treffen vereinbart. Der Benutzer kündigt um 12:20 Uhr an, ab 22:30 Uhr mit einem schwarzen Fahrzeug (Audi A2) vor einem Club zu warten; das Gegenüber sagt zu, sich unter dem Vorwand, „kurz an die frische Luft“ zu gehen, von der Begleitung zu entfernen. Der Chat belegt damit eine planvolle Kontaktanbahnung und die Verabredung eines konspirativen Treffens.


```
        "from": "rosanna_15",
        "to": "marty_17",
        "timestamp": "2026-03-06T12:15:00+02:00",
        "text": "Okay... Um wie viel Uhr bist du da?",
        "type": "text"
    },
    {
        "id": "msg_021",
        "from": "marty_17",
        "to": "rosanna_15",
        "timestamp": "2026-03-06T12:20:00+02:00",
        "text": "Ich steh ab 22:30 Uhr mit dem Auto vor dem Club. Schwarz, Audi A2, 5 T\u00fcren. Erkennst mich sofort.",
        "type": "text"
    },
    {
        "id": "msg_022",
        "from": "rosanna_15",
        "to": "marty_17",
        "timestamp": "2026-03-06T12:25:00+02:00",
        "text": "Perfekt. Ich sag meinen M\u00e4dels, ich muss kurz an die frische Luft. Dann komm ich raus.",
        "type": "text"
    },
    {
        "id": "msg_023",
        "from": "marty_17",
        "to": "rosanna_15",
        "timestamp": "2026-03-06T12:30:00+02:00",
        "text": "Super, freu mich riesig dich zu sehen. Bis heute Nacht! \ud83d\ude0a",
        "type": "text"
    },
    {
        "id": "msg_024",
        "from": "rosanna_15",
        "to": "marty_17",
        "timestamp": "2026-03-06T12:35:00+02:00",
        "text": "Bis sp\u00e4ter! \ud83d\ude0a",
        "type": "text"
```

Abbildung 18: Chatverlauf – Verabredung des Treffens (06.03.2026, 22:30 Uhr, schwarzer Audi A2)

9.6 Standort-Tracking (tracker.py / Locations.log)

Im Rahmen dieses Schrittes wurden das Skript tracker.py sowie die zugehörige, wiederhergestellte Log-Datei Locations.log ausgewertet. Das Skript wurde zunächst über seine Inode-Nummer im Verzeichnis tools lokalisiert und sein Inhalt gesichtet; anschließend wurde das Protokoll angezeigt:

```
cat Locations.log
```

Das Skript enthält eine Liste von Standortdaten mit Zeitstempeln und Ortsbezeichnungen und schreibt diese in Locations.log; zusammen mit dem Cron-Eintrag (9.3.2) ist damit ein automatisiertes, halbstündliches Protokollieren von Standorten belegt.



Abbildung 19: Ermittlung der Inode-Nummer für tracker.py (fls)

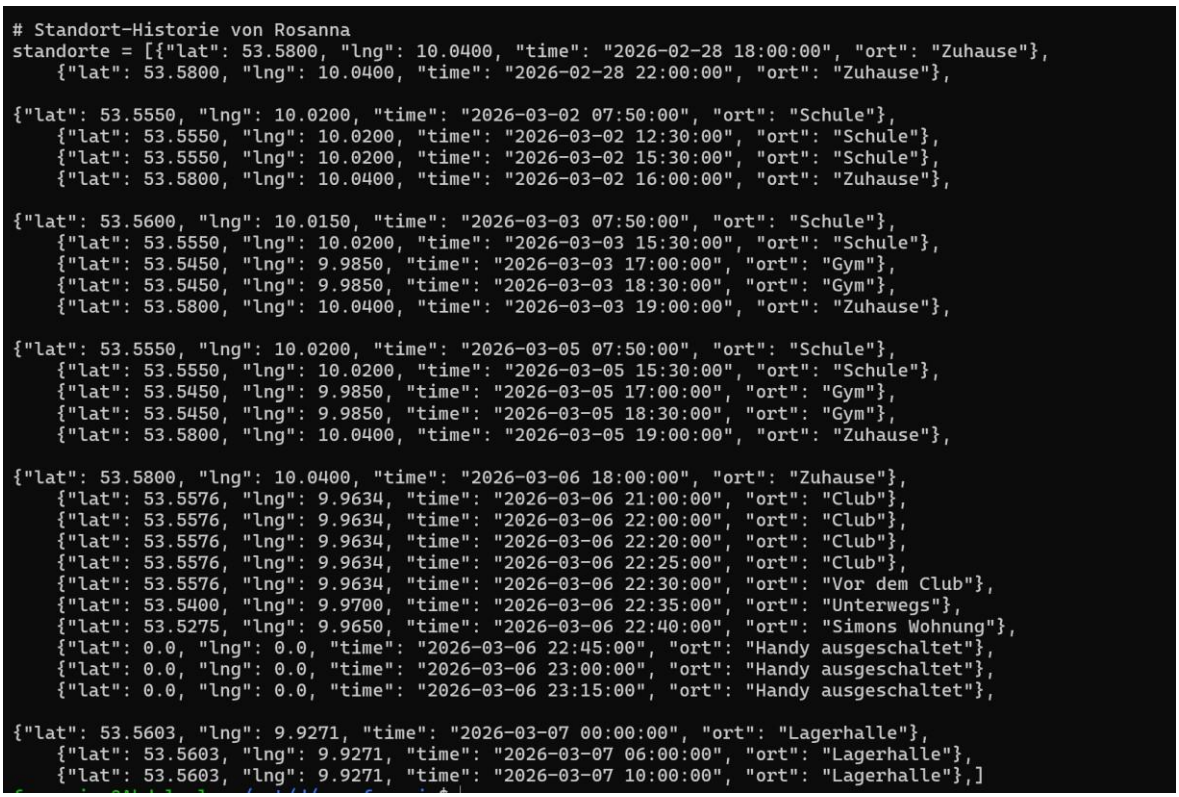


Abbildung 20: Inhalt des Skripts tracker.py mit hinterlegter Standortliste

Das wiederhergestellte Protokoll konkretisiert diesen Befund. Die aufgezeichnete Bewegung endet am Abend des 6. März 2026: Der letzte aktive Standort wird um 22:40 Uhr an der als „Simons Wohnung“ bezeichneten Position (53.5275, 9.965) verzeichnet. Unmittelbar danach protokolliert das Skript für 22:45, 23:00 und 23:15 Uhr ausschließlich „Handy ausgeschaltet“ (Koordinaten 0.0/0.0). Die letzten Einträge vom 7. März 2026 verweisen auf den Ort „Lagerhalle“ (53.5603, 9.9271). Der zeitliche Anschluss an das im Chat vereinbarte Treffen um 22:30 Uhr ist unmittelbar erkennbar.

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ cat /mnt/d/pea_fornsic/output/K7/recovered_carve/Locations.log
{"lat": 53.58, "lng": 10.04, "time": "2026-02-28 18:00:00", "ort": "Zuhause"}
{"lat": 53.58, "lng": 10.04, "time": "2026-02-28 22:00:00", "ort": "Zuhause"}
{"lat": 53.555, "lng": 10.02, "time": "2026-03-02 07:50:00", "ort": "Schule"}
{"lat": 53.555, "lng": 10.02, "time": "2026-03-02 12:30:00", "ort": "Schule"}
{"lat": 53.555, "lng": 10.02, "time": "2026-03-02 15:30:00", "ort": "Schule"}
{"lat": 53.58, "lng": 10.04, "time": "2026-03-02 16:00:00", "ort": "Zuhause"}
{"lat": 53.56, "lng": 10.015, "time": "2026-03-03 07:50:00", "ort": "Schule"}
{"lat": 53.555, "lng": 10.02, "time": "2026-03-03 15:30:00", "ort": "Schule"}
{"lat": 53.545, "lng": 9.985, "time": "2026-03-03 17:00:00", "ort": "Gym"}
{"lat": 53.545, "lng": 9.985, "time": "2026-03-03 18:30:00", "ort": "Gym"}
{"lat": 53.58, "lng": 10.04, "time": "2026-03-03 19:00:00", "ort": "Zuhause"}
{"lat": 53.555, "lng": 10.02, "time": "2026-03-05 07:50:00", "ort": "Schule"}
{"lat": 53.555, "lng": 10.02, "time": "2026-03-05 15:30:00", "ort": "Schule"}
{"lat": 53.545, "lng": 9.985, "time": "2026-03-05 17:00:00", "ort": "Gym"}
{"lat": 53.545, "lng": 9.985, "time": "2026-03-05 18:30:00", "ort": "Gym"}
{"lat": 53.58, "lng": 10.04, "time": "2026-03-05 19:00:00", "ort": "Zuhause"}
{"lat": 53.58, "lng": 10.04, "time": "2026-03-06 18:00:00", "ort": "Zuhause"}
{"lat": 53.5576, "lng": 9.9634, "time": "2026-03-06 21:00:00", "ort": "Club"}
{"lat": 53.5576, "lng": 9.9634, "time": "2026-03-06 22:00:00", "ort": "Club"}
{"lat": 53.5576, "lng": 9.9634, "time": "2026-03-06 22:20:00", "ort": "Club"}
{"lat": 53.5576, "lng": 9.9634, "time": "2026-03-06 22:25:00", "ort": "Club"}
{"lat": 53.5576, "lng": 9.9634, "time": "2026-03-06 22:30:00", "ort": "Vor dem Club"}
{"lat": 53.54, "lng": 9.97, "time": "2026-03-06 22:35:00", "ort": "Unterwegs"}
{"lat": 53.5275, "lng": 9.965, "time": "2026-03-06 22:40:00", "ort": "Simons Wohnung"}
{"lat": 0.0, "lng": 0.0, "time": "2026-03-06 22:45:00", "ort": "Handy ausgeschaltet"}
{"lat": 0.0, "lng": 0.0, "time": "2026-03-06 23:00:00", "ort": "Handy ausgeschaltet"}
{"lat": 0.0, "lng": 0.0, "time": "2026-03-06 23:15:00", "ort": "Handy ausgeschaltet"}
{"lat": 53.5603, "lng": 9.9271, "time": "2026-03-07 00:00:00", "ort": "Lagerhalle"}
{"lat": 53.5603, "lng": 9.9271, "time": "2026-03-07 06:00:00", "ort": "Lagerhalle"}
{"lat": 53.5603, "lng": 9.9271, "time": "2026-03-07 10:00:00", "ort": "Lagerhalle"}
```

Abbildung 21: Auszug aus der wiederhergestellten Log-Datei Locations.log (Ende: „Simons Wohnung“ 22:40, Handy ausgeschaltet, Lagerhalle 07.03.2026)

9.7 EXIF-Metadaten (lager.webp)

Der Shell-Verlauf (9.3.1) dokumentiert eine nachträgliche Veränderung der GPS-Metadaten der Datei lager.webp sowie das Löschen des Ausgangsbildes lager.webp_original. Dies begründete eine gezielte Metadatenanalyse der noch vorhandenen Bilddatei. Die Datei wurde über ihre Inode-Nummer lokalisiert, mit icat gesichert und ihre GPS-Felder mit exiftool ausgelesen:

```
exiftool -gpsposition -gpslatitude -gpslongitude lager.webp
```

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ fls -o 2203648 -rp /mnt/d/pea_fornsic/output/K7/FTK_Image_Dateien/simon.dd | grep "Downloa
ds/lager.webp$"
r/r 1837923:    home/simon-aman/Downloads/lager.webp
forensics@Abdulsalam:/mnt/d/pea_fornsic$
```

Abbildung 22: Ermittlung der Inode-Nummer für lager.webp (fls)

Die Auswertung bestätigte eingebettete GPS-Koordinaten (53°33’37,08” N, 9°55’37,56” E; entspricht 53.5603, 9.9271). Diese Koordinaten stimmen exakt mit dem abschließenden Standort „Lagerhalle“ aus dem Protokoll Locations.log überein und bestätigen diesen damit aus einer zweiten, technisch andersartigen Quelle. Der Beweiswert liegt nicht im sichtbaren Bildinhalt, sondern in den eingebetteten Standortinformationen.

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ exiftool -gpsposition -gpslatitude -gpslongitude /mnt/d/pea_fornsic/output/K7/lager.webp
GPS Position                    : 53 deg 33' 37.08", 9 deg 55' 37.56"
GPS Latitude                    : 53 deg 33' 37.08"
GPS Longitude                   : 9 deg 55' 37.56"
forensics@Abdulsalam:/mnt/d/pea_fornsic$
```

Abbildung 23: EXIF-Metadaten von lager.webp mit eingebetteten GPS-Koordinaten



Abbildung 24: Bilddatei lager.webp – zeigt das mutmaßliche Versteck

9.8 Bitcoin-Wallet (wallet.dat)

Ausgelöst durch die entsprechenden Einträge im Shell-Verlauf wurde das Verzeichnis bitcoin des Benutzers gesichtet. Die dort vorhandene Datei wallet.dat wurde über fls als regulärer, nicht gelöschter Eintrag (r/r 1837837) bestätigt und anschließend per icat aus dem System-Image gesichert; ihr Dateityp wurde geprüft und der Inhalt angezeigt:

```
icat -o 2203648 simon.dd 1837837 > wallet.dat
file wallet.dat cat wallet.dat
```

Entgegen dem Dateinamen handelt es sich nicht um eine binäre Wallet-Datenbank, sondern – wie file bestätigt – um eine ASCII-Textdatei. Sie enthält im Klartext eine Bitcoin-Adresse (1KQT8Hs1XZvjGJY9m2dmLhzj3cWj6jetSK) sowie den zugehörigen privaten Schlüssel im WIF-Format. Die Datei stellt die finanzielle Komponente des Sachverhalts dar. Ihre Bedeutung konkretisiert sich in Abschnitt 9.11: Die im entschlüsselten USB-Container gefundene Lösegeldforderung nennt exakt dieselbe Bitcoin-Adresse.

```
mon-aman/bitcoin/wallet.dat$"
r/r 1837837:    home/simon-aman/bitcoin/wallet.dat
forensics@Abdulsalam:/mnt/d/pea_fornsic$ icat -o 2203648 /mnt/d/pea_fornsic/output/K7/FTK_Image_Dateien/simon.dd 1837837 > /mnt/d/p
ea_fornsic/output/K7/wallet.dat
forensics@Abdulsalam:/mnt/d/pea_fornsic$ file /mnt/d/pea_fornsic/output/K7/wallet.dat
/mnt/d/pea_fornsic/output/K7/wallet.dat: ASCII text
forensics@Abdulsalam:/mnt/d/pea_fornsic$ cat /mnt/d/pea_fornsic/output/K7/wallet.dat
Bitcoin Wallet
Addresse: 1KQT8Hs1XZvjGJY9m2dmLhzj3cWj6jetSK
Private Key (WIF): 5J6n26xyco1Uru1a8Gk5DqfNtYeSnpVokApWY6Pky8WU3Rd1fTZ
forensics@Abdulsalam:/mnt/d/pea_fornsic$
```

Abbildung 25: Sicherung, Dateityp-Prüfung und Inhalt der Datei wallet.dat (icat, file, cat)

9.9 Arbeitsspeicheranalyse und Rückgewinnung des Kontopassworts

Der Shell-Verlauf (9.3.1) verwies auf das Öffnen eines verschlüsselten Containers sowie auf die Verwendung von steghide. Beide Vorgänge erfordern ein Kennwort, das nicht im Klartext protokolliert war. Da der Container zum Sicherungszeitpunkt geöffnet und das Benutzerkonto angemeldet war, mussten die zugehörigen Zeichenketten im Arbeitsspeicher vorliegen. Diese Arbeitsspeicheranalyse wurde daher vor der Steganografie (9.10) und der Container-Entschlüsselung (9.11) durchgeführt, da beide auf die hier hergeleiteten Kennwörter angewiesen sind.

Zunächst wurde mit Volatility 3 (Plugin linux.bash) die im Speicher verbliebene Befehlshistorie rekonstruiert:

```
vol -f "$RAM" linux.bash | grep -i -E "cryptsetup|luksOpen|mount /mnt/usb"
```

Die Ausgabe belegt, dass zum Sicherungszeitpunkt ein Datenträger mittels „sudo cryptsetup luksOpen /dev/sdb usb“ geöffnet und eingehängt war. Damit ist unabhängig bestätigt, dass ein verschlüsselter Container zur Laufzeit entsperrt war – die methodische Grundlage dafür, die zugehörigen Kennwörter aus dem flüchtigen Speicher herzuleiten statt anzunehmen.

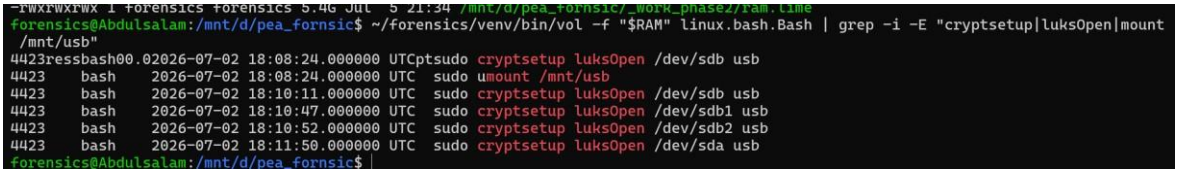


Abbildung 26: Rekonstruktion der Befehlshistorie aus dem RAM (Volatility 3, linux.bash) – cryptsetup luksOpen

Um die Kennwörter methodisch einwandfrei zu bestimmen, wurde nicht nach einem bekannten Wert gesucht. Stattdessen wurden mit strings alle druckbaren Zeichenketten des Speicherabbildes extrahiert, dedupliziert und als Kandidaten-Wortliste gespeichert:

```
strings -n 6 "$RAM" | sort -u > ram_wordlist.txt wc
-l ram_wordlist.txt
```

Die resultierende Wortliste ram_wordlist.txt umfasst 7.990.700 eindeutige Einträge (Dateigröße 211 MB). Sie bildet die Kandidatenbasis für die anschließende Validierung gegen unabhängige Prüfinstanzen; sie enthält keine Vorabkenntnis eines Kennworts.

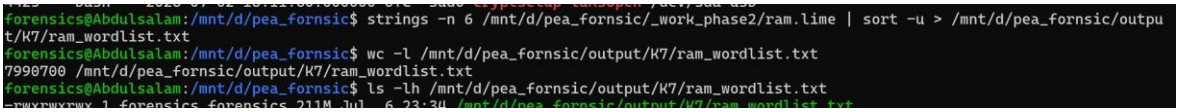


Abbildung 27: Extraktion aller Zeichenketten aus dem RAM-Abbild zur Kandidaten-Wortliste (7.990.700 Einträge, 211 MB)

Als Prüfinstanz für das Kontopasswort diente der in 9.2 gesicherte SHA-512-Hash aus /etc/shadow. Der Hash wurde über die Inode-Nummer der Datei lokalisiert und extrahiert:

```
fls -o 2203648 -rp simon.dd | grep "etc/shadow$"
icat -o 2203648 simon.dd 133865 | grep '^simon-aman:' > shadow.txt
```



Abbildung 28: Lokalisierung der Shadow-Datei im System-Image (fls, Inode 133865)

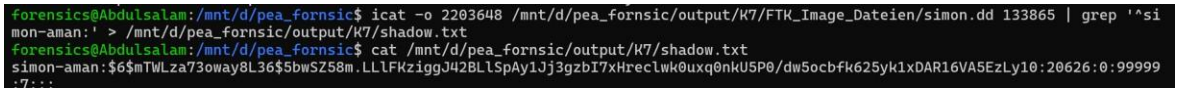


Abbildung 29: Extraktion des SHA-512-Passwort-Hashes des Kontos simon-aman (icat)

Anschließend wurde der Hash mit John the Ripper unter Verwendung der RAM-Wortliste geknackt:

```
john --wordlist=ram_wordlist.txt shadow.txt john
--show shadow.txt
```

Das Kontopasswort wurde erfolgreich zu SimonAman98 rückgewonnen. Da der Kandidat ausschließlich aus dem Speicherabbild stammt und der /etc/shadow-Hash ihn eindeutig bestätigt, ist das Kennwort ohne Vorwissen belegt. Ergänzend wurde die Übereinstimmung mit openssl passwd (SHA-512-crypt mit dem gespeicherten Salt) unabhängig nachvollzogen. Das so bestätigte Kontopasswort wird in 9.10 auf die Steganografie angewendet; die davon verschiedene ContainerPassphrase wird in 9.11 nach demselben Prinzip gegen den LUKS-Header validiert.

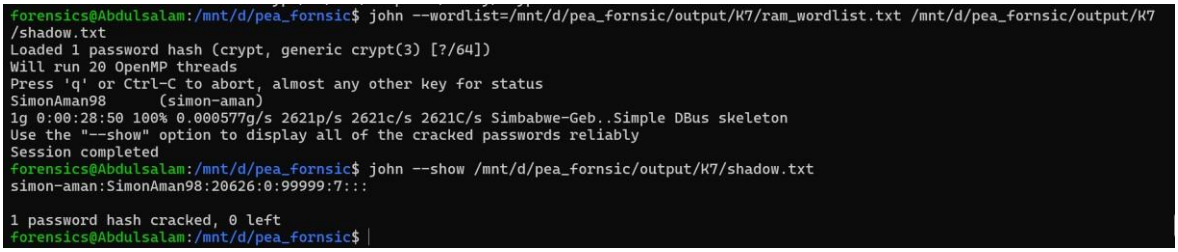


Abbildung 30: Rückgewinnung des Kontopassworts (John the Ripper, RAM-Wortliste) – simon-aman: SimonAman98

9.10 Steganografie (plan.jpg)

Der Shell-Verlauf (9.3.1) belegt, dass mittels steghide eine Textdatei in die Bilddatei plan.jpg eingebettet wurde. Die Bilddatei wurde zunächst über ihre Inode-Nummer lokalisiert und mit icat aus dem System-Image extrahiert:

```
fls -o 2203648 -rp simon.dd | grep plan.jpg icat
-o 2203648 simon.dd 1837852 > plan.jpg
```

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ fls -o 2203648 -rp /mnt/d/pea_fornsic/output/K7/FTK_Image_Dateien/simon.dd | grep "home/si
mon-aman/plan.jpg$"
r/r 1837852:    home/simon-aman/plan.jpg
```

Abbildung 31: Ermittlung der Inode-Nummer für plan.jpg (fls)

```
forensics@Abdulsalam:/mnt/d/pea_fornsic$ icat -o 2203648 /mnt/d/pea_fornsic/output/K7/FTK_Image_Dateien/simon.dd 1837852 > /mnt/d/p
ea_fornsic/output/K7/plan.jpg
forensics@Abdulsalam:/mnt/d/pea_fornsic$
```

Abbildung 32: Extraktion der Bilddatei plan.jpg aus dem System-Image (icat)



Abbildung 33: Bilddatei plan.jpg – unauffälliges Trägerbild der Steganografie

Für die Extraktion ist ein Kennwort erforderlich. Da Kennwörter erfahrungsgemäß mehrfach verwendet werden, wurde das in 9.9 aus dem Arbeitsspeicher hergeleitete Kontopasswort SimonAman98 angewendet. Zuvor wurde das fallspezifische Werkzeug steghide installiert.

```
sudo apt install steghide
```

```
ea_fornsic/output/K7/plan.jpg
forensics@Abdulsalam:/mnt/d/pea_fornsic$ sudo apt install -y steghide
```

Abbildung 34: Installation des Werkzeugs steghide

```
steghide info    plan.jpg -p SimonAman98
steghide extract -sf plan.jpg -p SimonAman98 cat
entführungsplan.txt
```

Der Zugriff war mit dem Kontopasswort erfolgreich und bestätigt damit zugleich die Wiederverwendung des Kennworts. steghide info weist eine eingebettete Datei entführungsplan.txt (154 Byte, Verschlüsselung rijndael-128, cbc, komprimiert) aus; steghide extract schreibt diese Datei heraus. Ihr Inhalt enthält eine konkrete Handlungsanweisung: das Abholen des Gegenübers am Club, das Verbringen zunächst in einen Keller und anschließend in ein Lager sowie die Wegnahme des Mobiltelefons. Diese Anweisung fügt sich widerspruchsfrei in die aus Chat und Standortdaten gewonnenen Erkenntnisse ein – insbesondere erklärt die Wegnahme des Mobiltelefons den zuvor beobachteten Eintrag „Handy ausgeschaltet“ im Standortprotokoll.

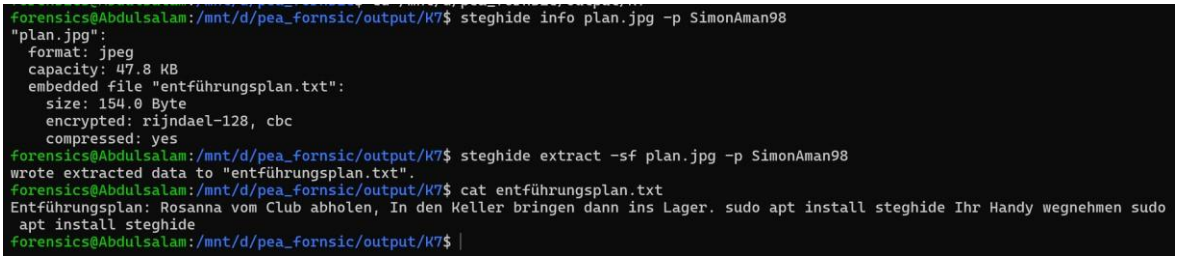


Abbildung 35: Steghide-Analyse und Extraktion der verborgenen Textdatei entführungsplan.txt (info, extract, cat)

9.11 USB-Container – Entschlüsselung (LUKS)

Das dritte Asservat, der verschlüsselte Container, wurde abschließend untersucht. Zunächst wurde das fallspezifische Werkzeug cryptsetup in der WSL2-Analyseumgebung installiert und seine Verfügbarkeit überprüft (cryptsetup 2.8.4).



Abbildung 36: Installation von cryptsetup über den Paketmanager apt



Abbildung 37: Versionsüberprüfung – cryptsetup 2.8.4 steht zur Verfügung

Der Container lag als virtuelle Festplatte im VHDX-Format vor und wurde über wsl --mount an die WSL2-Umgebung angebunden. Die Fehlermeldung WSL_E_USER_VHD_ALREADY_ATTACHED belegte, dass der Datenträger zu diesem Zeitpunkt bereits eingehängt war. Ein Vergleich der Blockgeräte vor und nach dem Einbinden mit lsblk zeigte den Container als neu erkanntes Gerät /dev/sde mit einer Kapazität von 4 GB.

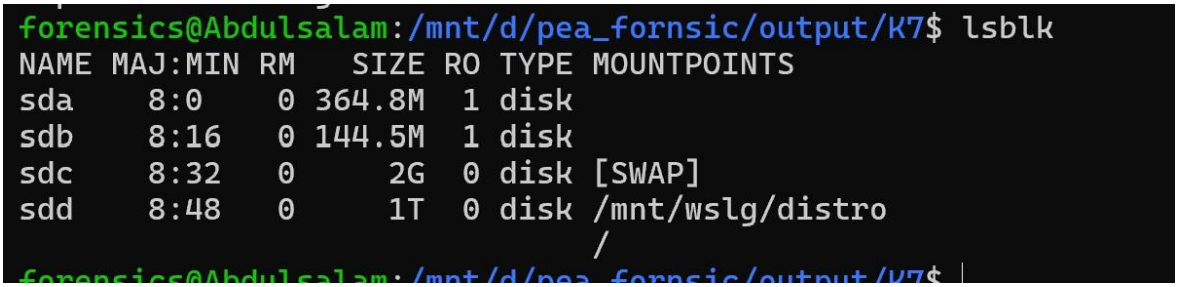


Abbildung 38: Blockgeräte vor dem Einbinden des virtuellen Datenträgers – der Container ist noch nicht sichtbar

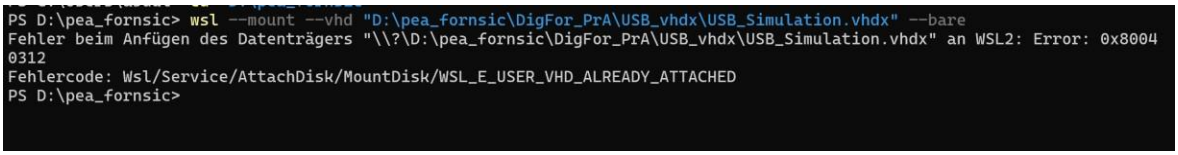


Abbildung 39: Anbinden der VHDX an die WSL2-Umgebung – die Meldung bestätigt, dass der Datenträger bereits eingehängt ist

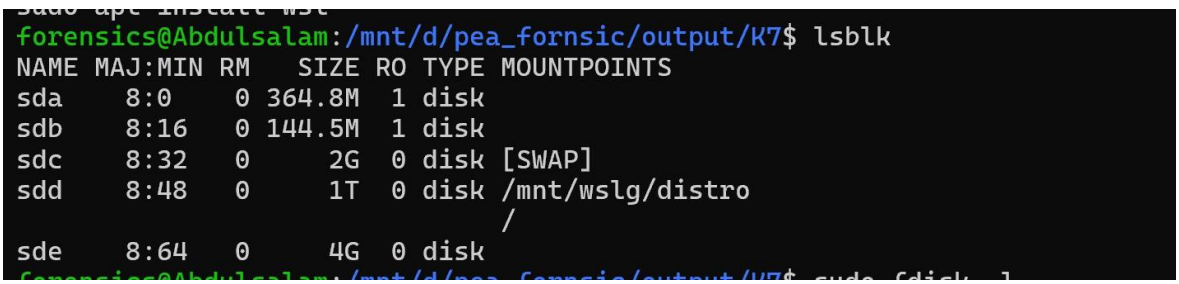


Abbildung 40: Blockgeräte nach dem Einbinden – der Container erscheint als /dev/sde (4 GB)

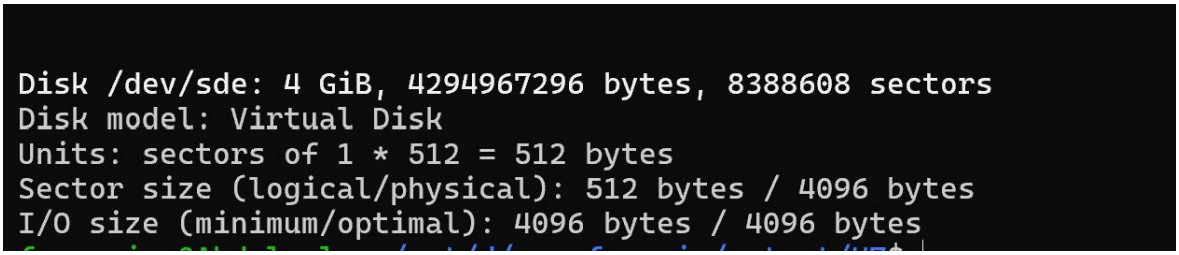


Abbildung 41: Technische Datenträgerparameter von /dev/sde – 4 GiB, virtuelle Festplatte, Sektorgröße 512/4096 Byte

Die Inspektion mit luksDump bestätigte eine LUKS2-Vollverschlüsselung des Datenträgers (UUID 4f769beb-bca8-495b-8219-a4521bca0803):

```
sudo cryptsetup luksDump /dev/sde | head
```

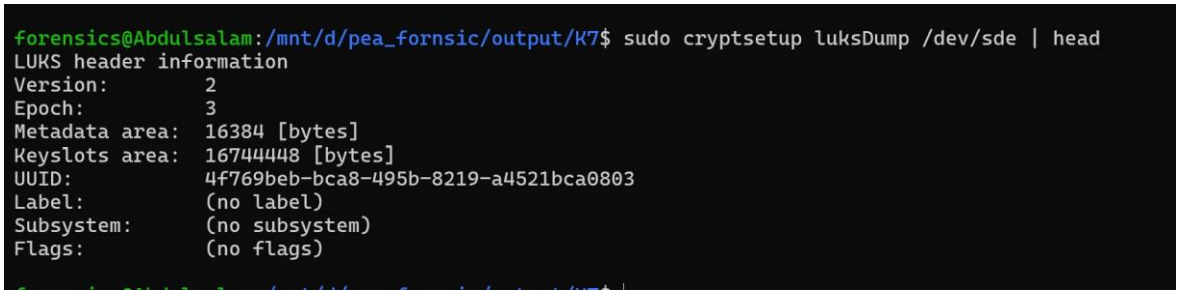


Abbildung 42: Ausgabe von cryptsetup luksDump – LUKS2-Verschlüsselung bestätigt

Die Passphrase wurde – analog zum Kontopasswort in 9.9 – nicht vorausgesetzt, sondern aus der RAM-Wortliste bestimmt. Als Prüfinstanz diente der LUKS-Header selbst: Jeder Kandidat aus ram_wordlist.txt wurde mit der Test-Funktion von cryptsetup gegen den Header geprüft, bis genau eine gültige Passphrase gefunden war.

```
while IFS= read -r p; do
  printf '%s' "$p" | sudo cryptsetup open --test-passphrase /dev/sde 2>/dev/null &&
{ echo "PASSPHRASE: $p"; break; }
done < ram_wordlist.txt
```

Die Validierung lieferte genau eine passende Passphrase: 12345abc!. Dieser Wert unterscheidet sich vom Kontopasswort (SimonAman98); beide wurden jedoch nach demselben Prinzip aus dem Arbeitsspeicher hergeleitet und gegen ihre jeweilige unabhängige Prüfinstanz bestätigt. Dass die Passphrase zum Sicherungszeitpunkt im Speicher vorlag, erklärt sich unmittelbar daraus, dass der Container zu diesem Zeitpunkt geöffnet war (vgl. 9.9).

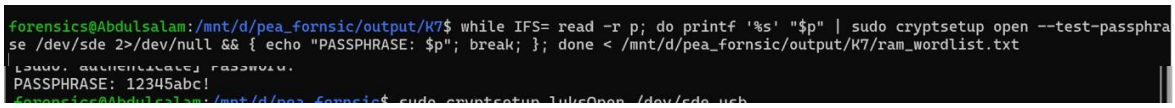


Abbildung 43: Validierung der RAM-Kandidaten gegen den LUKS-Header (cryptsetup --test-passphrase) – Treffer 12345abc!

Mit der validierten Passphrase wurde der Container geöffnet und lesend eingehängt. Ein zuvor mit einer abweichenden Passphrase durchgeführter Versuch schlug erwartungsgemäß fehl und dokumentiert die korrekte Funktion des LUKS-Schutzmechanismus.

```
sudo cryptsetup luksOpen /dev/sde usb
sudo mount /dev/mapper/usb /mnt/usb
ls -la /mnt/usb cat /mnt/usb/*.txt
```

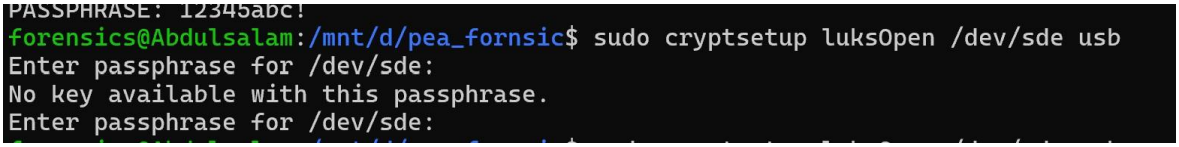


Abbildung 44: Fehlgeschlagener Entschlüsselungsversuch – eine nicht korrekte Passphrase wird abgelehnt

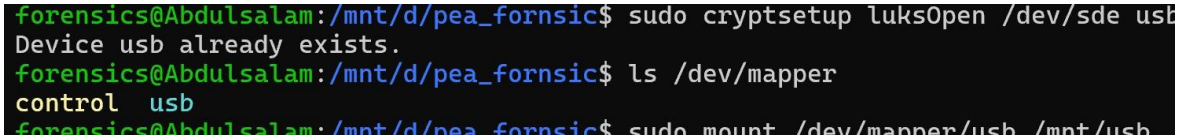


Abbildung 45: Öffnen des Containers (/dev/mapper/usb) und Einhängen des Dateisystems

Der geöffnete Container enthielt drei Dateien: lageraddresse.txt mit der Ortsangabe „Alte Ziegelei, Wedeler Landstraße 17, Hamburg“, loesegeld.txt mit einer Forderung über „200.000 Euro in Bitcoin“ an die Adresse 1KQT8Hs1XZvjGJY9m2dmLhzj3cWj6jetSK sowie die Bilddatei rosanna.jpg. Die genannte Bitcoin-Adresse ist identisch mit der Adresse aus der Datei wallet.dat (9.8); damit sind die finanzielle Spur und die Lösegeldforderung unmittelbar verknüpft. Diese Inhalte stellen die inhaltlich gewichtigsten Beweismittel dar und verbinden die zuvor gewonnenen Einzelbefunde (Ort, Forderung, Opferbezug).

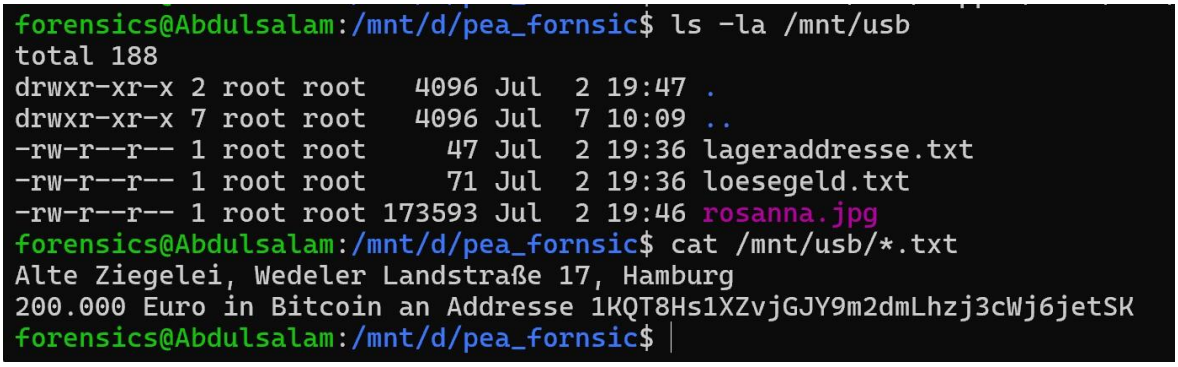


Abbildung 46: Dateiauflistung des entschlüsselten USB-Containers und Inhalt der Textdateien (Ortsangabe, Lösegeldforderung an dieselbe Bitcoin-Adresse wie wallet.dat)

9.12 Verwendete Werkzeuge und Untersuchungsumgebung

Die gesamte Auswertung wurde in einer dedizierten, schreibgeschützt betriebenen LinuxAnalyseumgebung auf Basis von Ubuntu unter dem Windows-Subsystem für Linux (WSL2) durchgeführt. Alle drei Asservate wurden ausschließlich lesend als forensische Kopien eingebunden; Schreibzugriffe auf die Originaldaten fanden zu keinem Zeitpunkt statt. Vor der inhaltlichen Analyse wurde die Unversehrtheit der Abbilder durch einen Abgleich neu berechneter Prüfsummen (unter anderem mittels sha256sum) mit den dokumentierten Referenzwerten bestätigt (Kapitel 8). Für die Erschließung des System-Images kam The Sleuth Kit zum Einsatz: mmls zum Auslesen der Partitionstabelle und zur Bestimmung des Partitionsoffsets, fsstat zur Bestätigung des ext4Dateisystems, fls zur rekursiven Verzeichnisauflistung, zur Ermittlung von Inode-Nummern sowie zum Nachweis vollständig entfernter Verzeichniseinträge (fls -rd) und icat zur verlustfreien Extraktion vorhandener Dateien anhand ihrer Inodes.

Da die gelöschten Dateien keine Verzeichniseinträge mehr besaßen, wurde für ihre inhaltsbasierte Rekonstruktion ein eigens erstelltes Python-Skript (carve_deleted.py) eingesetzt, das den unallozierten Speicherbereich anhand aus den vorherigen Befunden abgeleiteter Strukturmerkmale durchsuchte. Dieses Skript ist von The Sleuth Kit zu unterscheiden und stellte die beiden gelöschten Beweismittel parship_chatverlauf.json und Locations.log wieder her; ergänzend stellte strings die druckbaren Zeichenketten der Abbilder bereit.

Die inhaltliche Auswertung stützte sich auf spezialisierte Werkzeuge: Python (JSON-Aufbereitung des Chatverlaufs), exiftool (GPS-Metadaten von lager.webp), steghide (Extraktion der in plan.jpg verborgenen Textdatei) und Volatility 3 (Rekonstruktion der Befehlshistorie aus dem RAM mittels linux.bash). Die Rückgewinnung der Zugangsdaten folgte durchgängig dem Grundsatz, Kandidaten gegen unabhängige Prüfinstanzen zu validieren: strings erzeugte die RAM-Wortliste, John the Ripper knackte damit den /etc/shadow-Hash und openssl passwd bestätigte das Ergebnis (Kontopasswort); cryptsetup validierte die Container-Passphrase über die Funktion --test-passphrase gegen den

LUKSHeader und öffnete den Container anschließend mit luksOpen. Das Werkzeug hashcat stand als alternatives, GPU-gestütztes Verfahren zur Validierung gegen den gesicherten LUKS-Header zur Verfügung.
Eine vollständige Übersicht aller Werkzeuge und der jeweils tatsächlich verwendeten Befehle findet sich in der begleitenden Datei Fall4321_Forensic.xlsx.

9.13 Beweiskette und Ergebnis

Die Untersuchung führte ausgehend von drei zunächst unbeschriebenen Asservaten über die systematische Auswertung von Betriebssystem- und Benutzerartefakten zu einem geschlossenen Gesamtbild. Der Shell-Verlauf diente als zentrales Leitartefakt und wurde durch Cron-Konfiguration, zuletzt verwendete Dateien, Dateisystemauswertung, Metadaten- und Arbeitsspeicheranalyse jeweils unabhängig bestätigt. Sämtliche verwendeten Kennwörter wurden aus dem Speicherabbild hergeleitet und durch unabhängige Prüfinstanzen (/etc/shadow-Hash bzw. LUKS-Header) validiert, nicht vorausgesetzt.
Die Einzelbefunde greifen widerspruchsfrei ineinander: Der Chatverlauf belegt die Kontaktanbahnung und die Verabredung eines konspirativen Treffens; die wiederhergestellten Standortdaten führen unmittelbar von diesem Treffen zu einer Wohnadresse und weiter zu einem abgelegenen Ort; die manipulierten Bildmetadaten verweisen unabhängig auf denselben Ort; die verborgene Textdatei dokumentiert die Handlungsabsicht und erklärt die Abschaltung des Mobiltelefons; die Wallet-Datei und der entschlüsselte Container ergänzen die finanzielle und die inhaltliche Dimension, wobei die Lösegeldforderung dieselbe Bitcoin-Adresse nennt wie die Wallet-Datei. Die Arbeitsspeicheranalyse verbindet diese Stränge, indem sie das Kontopasswort (zugleich Steganografie-Passwort) und die davon verschiedene LUKS-Passphrase liefert.
Für die Bewertung der Beweiskette ist die Zuordnung des Systems zu einer Person maßgeblich. Das untersuchte Abbild wies genau ein reguläres Benutzerkonto auf – simon-aman –, das zugleich dem Hostnamen des Systems entspricht. Sämtliche belastenden Handlungen sind in der Shell-Historie dieses Kontos dokumentiert und wurden mit dessen Rechten ausgeführt.
Besonders belastend ist, dass dasselbe Kennwort, das das Anmeldekonto des Benutzers schützt, auch die steganografisch verborgene Datei öffnet: Die Personenbindung des Kontopassworts überträgt sich damit unmittelbar auf den verborgenen Tatplan. In der Zusammenschau führt die Beweiskette auf eine einzige handelnde Person; entlastende Artefakte, die einer anderen Person die Urheberschaft zuweisen würden, ergaben sich im Verlauf der Untersuchung nicht.

10 Zeitachse

Die nachfolgende Zeitachse fasst die Ereignisse des fiktiven Falls zusammen. Sie basiert auf den vorbereiteten Artefakten sowie den im Rahmen der Untersuchung gewonnenen Erkenntnissen. Die Spalte „Quelle“ dokumentiert, aus welchem Beweismittel bzw. Abschnitt der jeweilige Zeitpunkt abgeleitet wurde.

Datum	Uhrzeit	Ereignis	Quelle
28.02.2026	18:20 Uhr	Beginn der Kontaktaufnahme zwischen Tatverdächtigem und Opfer über die DatingPlattform	Kap. 9.5
06.03.2026	12:20 Uhr	Konkrete Vereinbarung des Treffens – Uhrzeit, Fahrzeug (schwarzer Audi A2) und Treffpunkt	Kap. 9.5
06.03.2026	22:30 Uhr	Vereinbartes Treffen vor einem Club	Kap. 9.5
06.03.2026	22:40 Uhr	Letzter aktiver Standort des Mobiltelefons an der Position „Simons Wohnung“	Kap. 9.6
06.03.2026	22:45–23:15 Uhr	Standortprotokoll verzeichnet ausschließlich „Handy ausgeschaltet“ (0.0/0.0)	Kap. 9.6
07.03.2026	00:00–10:00 Uhr	Dokumentierte Standorte an der „Lagerhalle“ (53.5603, 9.9271)	Kap. 9.6, 9.7

07.03.2026	—	Sicherstellung des Notebooks; RAM-Sicherung – USB-Container noch geöffnet, Kennwörter im Arbeitsspeicher herleitbar	Kap. 6, 9.9, 9.11

11 Beweiskette – Zusammenfassung

Die folgende Tabelle ordnet den einzelnen Beweismitteln die jeweilige forensische Erkenntnis und eine vorsichtige Einschätzung ihres Beweiswertes zu. Der Beweiswert ergibt sich maßgeblich aus dem Zusammenwirken mehrerer, voneinander unabhängiger Spuren.

Beweismittel	Forensische Erkenntnis	Beweiswert
Chatverlauf (parship_chatverlauf.json)	Kontaktanbahnung unter Pseudonym und Verabredung eines konspirativen Treffens	hoch
tracker.py + Locations.log	Automatisierte Standortüberwachung; letzter aktiver Standort an „Simons Wohnung“, anschließend Abbruch, Folgeort Lagerhalle	hoch
plan.jpg / entführungsplan.txt	Steganografisch verborgene Handlungsanweisung; mit dem Kontopasswort geöffnet	hoch
lager.webp (EXIF/GPS)	Manipulierte Metadaten mit GPS-Koordinaten; unabhängige Bestätigung des Endpunkts aus Locations.log	mittel
wallet.dat	Bitcoin-Adresse und privater Schlüssel; Adresse identisch mit der Lösegeldforderung im Container	mittel
USB-Container (LUKS)	Ortsangabe, Lösegeldforderung (dieselbe Bitcoin-Adresse) und Opferfoto	hoch
RAM-Abbild (Volatility 3)	Beleg des zur Sicherung geöffneten Containers; Herleitung von Kontopasswort und LUKS-Passphrase, validiert gegen /etc/shadow-Hash bzw. LUKS-Header	hoch

12 Rechtliche Einordnung

Auch wenn der Fall frei erfunden ist, lässt sich der unterstellte Sachverhalt grob folgenden Tatbeständen des deutschen Strafrechts zuordnen:

Tatbestand	Bezug zum Fall
§ 239 StGB – Freiheitsberaubung	Festhalten des Opfers in der Lagerhalle gegen dessen Willen
§ 235 StGB – Entziehung Minderjähriger	Verbringen einer 15-Jährigen von den Sorgeberechtigten weg
§ 253 StGB – Erpressung	Lösegeldforderung über eine Bitcoin-Adresse

Eine tiefere strafrechtliche Würdigung ist nicht Gegenstand dieser technischen Ausarbeitung. Die Zuordnung dient ausschließlich der groben rechtlichen Einordnung des fiktiven Falls und ersetzt keine juristische Bewertung.

13 Fazit

Im Rahmen dieser Projektarbeit wurde ein realitätsnaher forensischer Fall entwickelt, forensisch gesichert und ausgewertet. Für die Sicherung wurden Datenträger- und Arbeitsspeicherabbilder erstellt und ihre Integrität durch Hashwertbildung nachgewiesen. Die eigentliche Auswertung der gesicherten Artefakte ist in Kapitel 9 dokumentiert.

Die Untersuchung konnte die vorbereiteten digitalen Artefakte nachweisen und auswerten. Durch die Kombination aus Dateisystemanalyse, Wiederherstellung gelöschter Dateien, Arbeitsspeicherforensik

sowie der Auswertung versteckter und eingebetteter Informationen ließ sich der fiktive Tathergang schlüssig und weitgehend widerspruchsfrei rekonstruieren.
Besonders deutlich wird der Beweiswert dort, wo sich mehrere unabhängige Spuren gegenseitig bestätigen: etwa die GPS-Koordinaten aus den Bildmetadaten und der abschließende Eintrag im Standortprotokoll oder die identische Bitcoin-Adresse in wallet.dat und in der Lösegeldforderung. Von zentraler Bedeutung sind die beiden aus der RAM-Wortliste hergeleiteten und gegen unabhängige Prüfinstanzen validierten Kennwörter: das Kontopasswort SimonAman98, das zugleich die steganografisch verborgene Datei öffnet, sowie die davon verschiedene LUKS-Passphrase 12345abc!, die den USB-Container entschlüsselt. Die Beweismittel ergänzen sich zu einer schlüssigen, in sich widerspruchsfreien Beweiskette, die sich dem Benutzerkonto des Tatverdächtigen zuordnen lässt.

13.1 Grenzen der Untersuchung und Reproduzierbarkeit

Die Untersuchung wurde vollständig innerhalb einer virtualisierten Laborumgebung durchgeführt. Dadurch konnten wesentliche Schritte einer digitalen forensischen Untersuchung realitätsnah nachgebildet werden. Gleichzeitig ersetzt eine virtuelle Umgebung keine reale Sicherstellung physischer Datenträger. Aspekte wie der Einsatz eines Hardware-Write-Blockers, physische Datenträgerdefekte oder herstellerspezifische Besonderheiten konnten nicht berücksichtigt werden. Da sämtliche Artefakte gezielt innerhalb der virtuellen Umgebung vorbereitet wurden, ist der Fall reproduzierbar. Eine erneute Untersuchung derselben Images führt bei unveränderten Ausgangsdaten zu identischen Ergebnissen. Die durchgängige Hashwertbildung stellt sicher, dass sämtliche Analysen auf unveränderten Beweismitteln beruhen und die Integrität der Abbilder jederzeit nachvollziehbar bleibt.

13.2 Einschränkungen und Ausblick

Der Umfang des Falls wurde bewusst auf die im Praktikum behandelten Themen begrenzt. Neben dem Systemabbild, dem verschlüsselten USB-Container und dem Arbeitsspeicherabbild wurden keine weiteren Datenquellen wie Mobilfunkdaten, Cloud-Speicher oder Netzwerkmitschnitte berücksichtigt. In einer realen Ermittlung würden solche Informationen häufig zusätzliche Erkenntnisse liefern.
Eine mögliche Erweiterung wäre daher die Einbindung weiterer digitaler Beweismittel, beispielsweise mobiler Endgeräte, Cloud-Dienste oder zusätzlicher Netzwerkkommunikation. Ebenso könnte die Untersuchung um weitere Bereiche der digitalen Forensik, etwa die Analyse eines Smartphones oder die Auswertung von Netzwerkverkehr, ergänzt werden.

14 Quellenverzeichnis

bitaddress.org. Bitcoin Wallet Generator. https://www.bitaddress.org. Verwendet zur Generierung der fiktiven Bitcoin-Adresse und des privaten Schlüssels für die Datei wallet.dat.
FakeNameGenerator. Zufallsgenerator für fiktive Namen. https://www.fakenamegenerator.com. Verwendet zur Generierung der fiktiven Personennamen im Rahmen der Fallerstellung.

Abkürzungsverzeichnis

Abkürzung	Bedeutung
RAM	Random Access Memory (Arbeitsspeicher)
LUKS	Linux Unified Key Setup (Festplattenverschlüsselung)
dd	Datenformat für 1:1-Sektorabbilder (Raw Image)
FTK	Forensic Toolkit (hier: FTK Imager)
MD5 / SHA1 / SHA256	Kryptografische Hashfunktionen zur Integritätsprüfung
VM	Virtuelle Maschine
VHDX	Virtual Hard Disk (Hyper-V-Festplattenformat)
GPS	Global Positioning System
EXIF	Exchangeable Image File Format (Bild-Metadaten)
WSL2	Windows-Subsystem für Linux (Version 2)
AVML	Acquire Volatile Memory for Linux
LiME	Linux Memory Extractor (Format des RAM-Abbildes)
WIF	Wallet Import Format (Kodierung privater Bitcoin-Schlüssel)

15 Anhänge

Anhang A (Nextcloud)

URL: https://nextcloud.th-deg.de/apps/files/files?dir=/DigFor_PrA
Struktur: Artefakte Lager.webp
Map.jpg
Parship_chatverlauf.json
Tracker.py
Forensik_Final_Results
build_tools_xlsx.py
carve_deleted.py
entführungsplan.txt
lager.webp plan.jpg
ram_wordlist.txt
recovered_carve.zip
recovered_icat.zip
rosanna.jpg shadow.txt
wallet.dat
Gutachten
Forensischer_Bericht_Fall4321_AlHendi_Alhammoud_Heinl.pdf
Evidence_Forms_Fall4321.pdf
SimonAman_Ubuntu_System
FTK_Image_Dateien.zip
FTK_Images_hashes.png
Ram_dump_hash.png

Ram_dump.lime.compressed

USB_vhdx

USB_Simulation.vhdx

USB_Simulation_hash.png

Fall_Vorbereitung_und_ImagesHerstellung.pdf